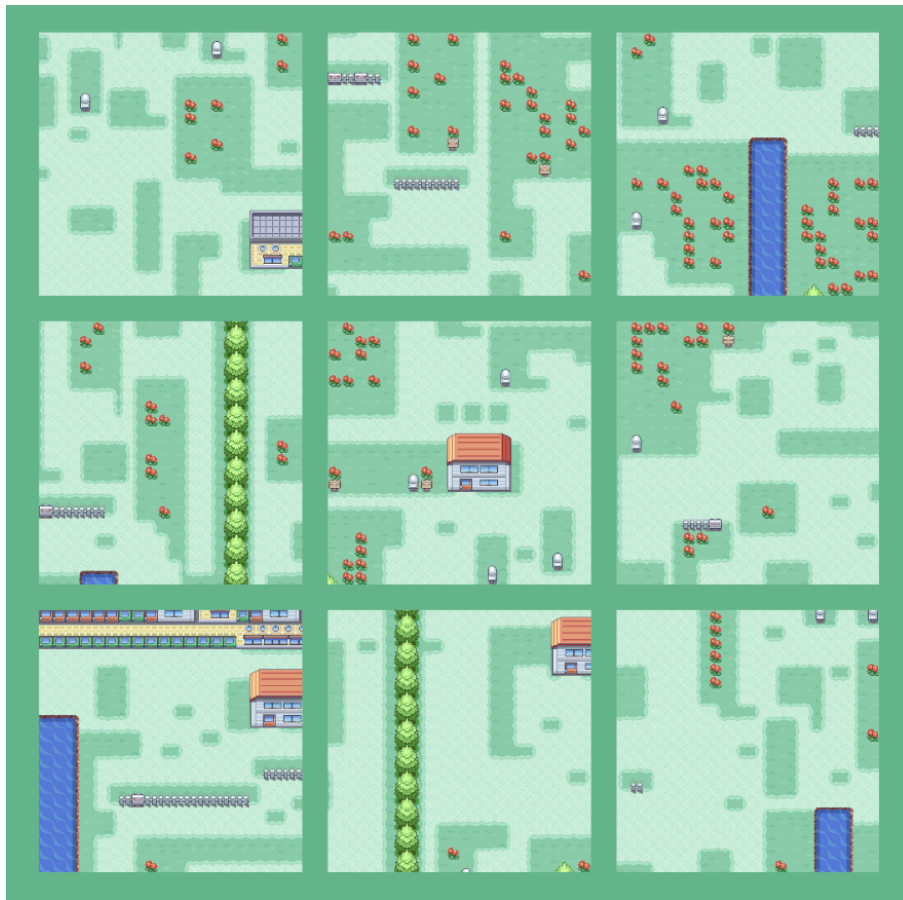


Procedural Level Generation through Constrained MAP-Elites:

Optimizing Reliability and Consistency in
Wave Function Collapse



Oliver Juhl Friis Hansen (oljh@itu.dk)
William Skou Heidemann (wihe@itu.dk)

IT University of Copenhagen
Master's Thesis for MSc in Games
KISPECI1SE

June 1, 2026

Abstract

Procedural Content Generation (PCG) can be a helpful tool in creating content without the need for human input. However, PCG must balance the generated content between generating too similar content, leading to boredom, or too random content, leading to chaos.

Wave Function Collapse (WFC) is a constraint-solving algorithm often used in PCG. WFC is a stochastic algorithm, where it uses randomness to generate different pieces of content every time. This makes it challenging to use for PCG, as it may introduce too much randomness and become inconsistent. Further, WFC can reach an unsolvable state, where it can no longer reach a feasible level, making it unreliable.

In this thesis, we use WFC as a 2D tilemap level generator, and we use Search-Based Procedural Content Generation to generate input for WFC. In particular, we use Constrained MAP-Elites (CME) to find input tile weights that make WFC more consistent and reliable. We measure consistency as the similarity of levels, and we measure reliability as the percentage of levels that avoid contradictions.

For this purpose, we use two synthetic example domains, one without any constraints and another with many. Finally, we use a tilemap from *Pokémon Fire Red* (Game Freak, 2004), as a case-study for a real-world tilemap.

Our experiments show that the mutation of input weights to WFC can illuminate a behavior space. They also show that CME drives the population towards better reliability and consistency over time. However, our experiments also illustrate that creating levels with certain very specific characteristics is too difficult or even impossible for CME to find solutions for.

Acknowledgements

First and foremost, we would like to thank our supervisor, Djordje Grbic, for his guidance and support throughout the project.

We would also like to thank our peers and friends for sharing their knowledge and keeping us company throughout the creation of this thesis.

Declaration of Generative AI usage

We have developed the technical product using the JetBrains Rider IDE, which features full-line code completion (JetBrains, n.d.), which we have used for auto-completing some lines of code after reviewing their correctness.

We have used Google Gemini for debugging code, asking clarifying questions about other sources, and drafting boilerplate Latex (such as inserting figures and tables) (Google, n.d.).

All text and code in the delivered product is produced and written by us, and generative AI has only been used for assistance.

Contents

1	Introduction	1
1.1	Boredom vs. Chaos	1
1.2	Taxonomy of PCG	2
1.3	Wave Function Collapse	3
1.4	Contributions and research	3
1.4.1	Research Questions	4
1.5	Thesis Outline	4
2	Background	5
2.1	Procedural Content Generation	5
2.2	Wave Function Collapse	6
2.2.1	Constraint Solving	7
2.2.2	Model Synthesis	7
2.2.3	The challenge with WFC	8
2.3	Search-Based Procedural Content Generation	10
2.3.1	Quality-Diversity	11
3	Methodology	13
3.1	Project Structure	13
3.2	Wave Function Collapse	14
3.2.1	Data Oriented Design	15
3.2.2	BitArray	16
3.3	MAP-Elites	16
3.3.1	Constraining MAP-Elites	17
3.3.2	Generic Implementation	18
3.3.3	Evolutionary Strategies	18
3.3.4	Random Initialization	21
3.3.5	Random Variation	21
3.3.6	Evaluation	22
3.3.7	Key in the archive	26
3.3.8	Archive Explorer	26
3.4	Tilemap Analysis	27
3.5	Statistics	27

4	Experiments	29
4.1	Domains	30
4.1.1	Letter Domain	30
4.1.2	Arrow Domain	31
4.1.3	Pokémon Domain	31
4.2	Standard Deviation	33
4.2.1	Letter Domain	34
4.2.2	Arrow Domain	34
4.2.3	Pokémon Domain	35
4.3	Domain Difficulty	36
4.3.1	Letter Domain	36
4.3.2	Arrow Domain	38
4.3.3	Pokémon Domain	39
4.4	Evaluation Iterations	42
4.5	Variation Behavior	43
4.6	Mutation Strategies	45
5	Results	48
5.1	Illumination of Behavior Space	48
5.2	Reliability of Wave Function Collapse	49
5.3	Consistency of Wave Function Collapse	50
6	Conclusion	52
6.1	Contributions	52
6.2	Future Work	52
6.2.1	Mutation Strategies	53
6.2.2	Other Genotypes	53
6.2.3	Mixed-Initiative Design	53

List of Figures

2.1	An example of generating a level with WFC on a 2x2 grid. The input consist of 9 tiles and a set of adjacency rules. For simplicity, two tiles can be adjacent if they have similar color values along the border. . . .	7
2.2	Comparison between cell selection heuristics. Source: Gumin, 2016. . .	8
2.3	WFC resulting in a contradiction.	9
3.1	An overview of our modules and their dependencies.	13
3.2	Memory layout of array of structures vs structure of arrays.	15
3.3	Black dots are behaviors and white dots are the average of the black dots connected to it. (1) has a large distance from the mean giving it a bad fitness score. (2) has a smaller distance from the mean giving it a better fitness in the same cell. Therefore (2) replaces (1) in the archive.	24
3.4	Different smoothing factors of fitness activation function e^{-kx} . $k = 1$ (blue), $k = 5$ (black) and $k = 20$ (red). Created with Desmos Graphing Calculator (Desmos, n.d.).	25
3.5	Screenshot from the <i>Archive Explorer</i>	27
4.1	A tileset of 5 letter-tiles that will always produce a feasible solution. . .	30
4.2	9 example levels generated by WFC using the Letter domain.	30
4.3	A tileset of 4 arrow-tiles and an X-tile.	31
4.4	9 example levels generated by WFC using the Arrow domain.	31
4.5	Tilemap of Pallet Town from <i>Pokémon Fire Red</i> (Game Freak, 2004) . . .	32
4.6	Example levels generated by WFC using the Pokémon domain.	32
4.7	The construction of an infeasible level. Purple tiles are used to show a contradiction.	33
4.8	Impact of σ on <i>global reliability</i> and <i>convergence rate</i> for the Letter domain	34
4.9	Impact of σ on <i>global reliability</i> and <i>convergence rate</i> for the Arrow domain.	35
4.10	Impact of σ on <i>global reliability</i> and <i>convergence rate</i> for the Pokémon domain.	35
4.11	Behavior space of the archive generated for the Letter domain.	37
4.12	QD score and coverage of the archive for the Letter domain.	37
4.13	Behavior space of the archive generated for the Arrows domain. . . .	38
4.14	Both levels use exactly 2 tiles.	39

4.15	QD score and coverage of the archive for the Arrow domain.	39
4.16	Behavior space of the archive generated for the Pokémon domain. . .	40
4.17	Comparison of levels produced by genotypes in opposite ends of the flower-tile axis.	40
4.18	Feasibility graph of the evolution of the archive generated from the Pokémon domain.	41
4.19	QD score and coverage of the archive for the Pokémon domain.	41
4.20	The impact of different values of M on fitness and feasibility.	42
4.21	Behavior space comparison of the two variation behaviors.	44
4.22	QD score of the two variation behaviors.	44
4.23	Behavior space plot of the four mutation strategies.	46
4.24	Feasible and Infeasible Population in the archive over time.	47

List of Tables

3.1	Benchmark results comparing WFC implementations using <i>HashSet</i> vs. <i>BitArray</i> for set operations.	16
3.2	Overview of different mutation strategies.	22
4.1	Overview of the parameters used for our experiments.	29
4.2	Evaluation Iteration impact on running times and QD score. All experiments use $I = 5000$ with no convergence.	43
4.3	Results for different mutation strategies. Refer to table 3.2 for details on mutation strategies.	45

1 Introduction

As video games grow in scale and complexity, they require more content. It is not always viable for small teams to create all this content manually, especially for games where new content is needed every time the game is played.

To address this problem, it is desirable to generate content automatically through an algorithm. This is also known as Procedural Content Generation (PCG). Shaker et. al. defines PCG as "the algorithmic creation of game content with limited or indirect user input" (Shaker et al., 2016, Ch. 1). With this approach a large volume of content can be generated quickly, even at runtime. PCG can be used to generate most things within a game, such as levels, equipment, quests, music and much more. However, this comes with its own challenges, one of which we describe in the following section.

1.1 Boredom vs. Chaos

Backus, 2017 highlights a central problem with PCG. Humans are good at pattern recognition, and content generation algorithms are often pattern-producing machines. The aim of a generator then is to mask the fact that it produces patterns. This places generators on a spectrum. On the one end, the generated content is so similar that it becomes boring. On the other end, the generated content becomes so chaotic and nonsensical that it becomes unplayable. A generator between these extremes produces content that "pleasantly surprises" (Backus, 2017) the user. Backus calls this balance *boredom vs. chaos*.

Backus suggests that you find this balance by designing content before the generator. The content design can inform how the generator should be adjusted. In other cases, it may be beneficial to build a generator and observe what it generates. This can be useful when it is not clear how to get the desired result for the generator. To facilitate this, Backus suggests using search algorithms that use the generator to find interesting solutions efficiently (Backus, 2017). This is also called meta-optimization, since the goal is to optimize the input to the generator, rather than its output directly. To further examine meta-optimizers, we present a brief taxonomy of PCG.

1.2 Taxonomy of PCG

Shaker et al., 2016 present a taxonomy of PCG. They distinguish between *deterministic* and *stochastic* generators. Deterministic generators always generate the same output, given the same input. An example of a deterministic generator is the galaxy generator in *Elite* (Braben & Bell, 1984), which was implemented to save space on the disk. In this sense, the galaxy generator is a decompression algorithm. A stochastic generator, on the other hand, can generate a different output given the same input. An example of a stochastic algorithm is the dungeon generation algorithm in *The Binding of Isaac* (Edmund McMillen and Florian Himsl, 2011). From a starting room, the generator determines how many rooms to create. Then it picks rooms randomly from a pool of possible rooms, creating new layouts every time the game is played.

Shaker et al. also distinguish between methods of generation: *Constructive* vs. *generate-and-test*. For constructive methods, the generator runs once. For generate-and-test methods, the generator produces an output that may be accepted or discarded (Shaker et al., 2016).

One subgroup of generate-and-test methods is *Search-Based PCG* (SB-PCG) (Togelius et al., 2011). While generate-and-test either accepts or discards a solution, SB-PCG uses an *evaluation* (or *fitness*) score to determine how good a solution is.

Most SB-PCG is evolutionary, which means that it runs for several iterations or *generations*. It uses the fitness values from previous iterations to rank or select individuals as the basis for the next iteration. Over time, SB-PCG finds high quality input to the generator, according to the chosen evaluation approach.

A specific kind of SB-PCG is *quality-diversity* (QD) (Gravina et al., 2019). QD methods emphasize both the quality aspect described earlier and the diversity of solutions. Backus, 2017 suggests Novelty Search for meta-optimization of generators, which is a QD algorithm. QD algorithms find diversity by exploring a space of *behaviors* — certain characteristics, which the user wants the generated content to exhibit to varying degrees. An example behavior for a level could be the size of the level, the number of treasure chests in the level, or the difficulty of the level. The combination of the behaviors is called the *behavior space*.

In this thesis, we explore how to apply a QD algorithm to control the input of a stochastic level generation algorithm to produce a diverse set of high quality content consistently. In the following section, we introduce one such level generation algorithm.

1.3 Wave Function Collapse

Level generation is a frequent application of PCG. There are many ways to generate levels. We choose the Wave Function Collapse algorithm (WFC) as a case study, which is a constraint-solving algorithm. The input given to the algorithm is a set of tiles, adjacency rules, and weights and the output is a level. The output uses only the tiles in the input. The rules define which tiles are allowed next to each other. Lastly, the weights influence what tile the algorithm picks from a set of options to place in the level (Gumin, 2016).

WFC is an interesting algorithm, since it is stochastic and is not guaranteed to finish. This can make it hard to use, because it can reach a state which cannot be completed due to its constraints. In WFC, this state is called a *contradiction*.

Because WFC is a stochastic algorithm, it can be hard to guarantee what kinds of output levels the algorithm produces. Previous work has used a genetic algorithm to modify the input tilemap of a WFC implementation that derives the rules from a tilemap (Brellas, 2021).

Our research aims to control the stochastic nature of WFC by using QD to perform mutations directly on the set of input weights.

1.4 Contributions and research

In this thesis, we propose a strategy for optimizing and illuminating the behavior space of a stochastic level generation algorithms. In particular, our research uses WFC as a case study. We use a set of tiles, rules, and weights as input for WFC to generate levels. By changing the weights through the QD algorithm *MAP-Elites*, we illuminate a behavioral space where the generated levels exhibit specific characteristics. At the same time, MAP-Elites optimizes WFC to avoid contradictions so that it completes more often (reliability) and to generate levels that land in the expected area of the behavior space (consistency). In short, we generate input to WFC, so that it generates levels that land somewhere in-between boredom and chaos.

Further, we introduce a tool to explore the archive of solutions generated by MAP-Elites called the *Archive Explorer*. Through this tool, a game designer can choose which part of the behavior space they wish to explore. This tool can be used in mixed-initiative design, where designers can use the archive as inspiration and a baseline.

For our experiments, we introduce two toy domains for WFC to evaluate our methods. Then we use a real-world example of a tileset from Pokémon Fire Red (Game Freak, 2004) to see how our methods handle production ready tilesets.

1.4.1 Research Questions

We will evaluate the following research questions:

- Can the mutation of weights for the WFC algorithm illuminate a behavior space?
- Can QD improve the reliability of the WFC algorithm?
- Can QD improve the consistency of the WFC algorithm?

1.5 Thesis Outline

In chapter 2, we review related work and the theoretical background for our work. In chapter 3, we present our methods, approach and implementations. In chapter 4, we present our experiments and show the results. In chapter 5, we relate these results to our research questions and evaluate whether we succeed. In chapter 6, we summarize the thesis and discuss avenues for further work.

2 Background

In this chapter, we expand on the field presented in the introduction. We present related work in the field and what other methods have been used to approach our problem. We then place our research question in relation to the field at large.

Firstly, we present Procedural Content Generation. Then, we introduce the Wave Function Collapse Algorithm. Lastly, we introduce the field of Search-Based Procedural Content Generation.

2.1 Procedural Content Generation

Procedural Content Generation (PCG) differs from manual content design, since it delegates the responsibility of creating assets or environments to software processes. A generator must not only function efficiently enough to meet the technical constraints of the game but also ensure that the produced content is playable. In addition, the content must be diverse to avoid repetition. The produced content may sometimes be used as a starting point for designers as something to expand upon or simply as inspiration rather than direct game play generation (Shaker et al., 2016, p. 3).

PCG has seen many applications in the games industry. Games like *Minecraft* (Mojang Studios, 2011) and *No Man's Sky* (Hello Games, 2016) use PCG to create entire game worlds, landscapes and variations of natural biomes. Using PCG to create large pieces of content like this shows the capabilities of this technique. On the other end of the spectrum we have games like *Townscaper* (Oskar Stålberg, 2021) and *Tiny Glade* (Pounce Light, 2024) where the *player* is given the power to procedurally generate their own content in real time. In these games, the player continually provides direction to the generator, rather than pressing the generate button once.

Further, there are games like *The Binding of Isaac* (Edmund McMillen and Florian Himsl, 2011) and *Spelunky* (Mossmouth, Derek Yu, Blitworks, 2008) where levels are procedurally generated as the player progresses through the game. These level generators are stochastic, since they give a different output every time they are run. This effectively provides players with near infinite re-playability.

PCG comes with its own challenges, however. Backus, 2017 highlights the act of balancing *boredom vs. chaos* as an essential challenge. Backus offers a way to approach this problem by introducing constraints to limit the kind of content you can generate. These constraints then shape the content design.

Similarly, Grey, 2017 explains, how game designers can create a controlled experience by introducing a set of constraints to the generator function. Level generation is a classic example of procedural content generation where a set of design constraints dictate how a procedural algorithm may (or may not) generate content. An example of a constraint may be that the generated level must have an entrance and an exit. Another might be that there must be a path between the entrance and the exit. Grey, 2017 also warns that introducing too many constraints may lose the variety PCG brings with over-constrained, "samey" levels. A balance between boredom vs. chaos.

That being said, constraints are an important part of content design. One algorithm that utilizes constraints to generate content is the Wave Function Collapse algorithm. The following section introduces the algorithm and situates it within the broader field of constraint solving.

2.2 Wave Function Collapse

Wave Function Collapse (WFC) is an algorithm created by Maxim Gumin (Gumin, 2016). The algorithm can generate an output according to some constraints from the input. WFC can generate a wide range of content, including 2D images (Gumin, 2016), 3D models (Oskar Stålberg, 2021) and even poems (O’Leary, 2026). For this thesis, we will focus on Wave Function Collapse as it applies to 2D images.

Gumin introduces two different models of WFC. One is the Simple Tiled Model and the other is the Overlapping Model. They differ mostly in the inputs. The Simple Tiled Model takes as input a set of tiles and a set of adjacency rules that define what tiles can go next to each other. The algorithm outputs a grid of tiles that are consistent with the rules provided in the adjacency rules.

The algorithm repeatedly chooses a cell in the grid and decides which tile to put in that cell. The algorithm will prefer to choose a cell where it is more certain which tile will be selected. After deciding on a tile for a cell, a ripple effect is triggered where neighbor cells are updated to remove any tiles that no longer follow the rules. This process continues until the entire grid is populated, or a contradiction occurs where a cell has no valid options. See figure 2.1 for an example of the WFC procedure creating a level.

Further, the Simple Tiled Model can use *symmetries*. A symmetry defines how a tile can be rotated in relation to other tiles.

The Overlapping Model takes an image as input and then extracts tiles and adjacency rules by analyzing the image (Gumin, 2016). The overlapping model overlaps tiles when generating the output, using only the center part of each tile.

For this thesis, we want to generate simple levels for 2D games. Therefore we use

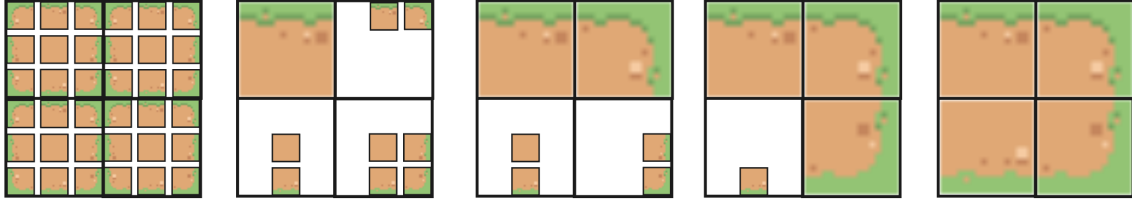


Figure 2.1: An example of generating a level with WFC on a 2x2 grid. The input consist of 9 tiles and a set of adjacency rules. For simplicity, two tiles can be adjacent if they have similar color values along the border.

the Simple Tiled Model and use a different system for deriving symmetries. This will be explained further in chapter 3.2.

No matter the model of WFC, it uses constraints in the form of adjacency rules to generate an output. To understand the motivation behind this approach, we briefly introduce the field of constraint solving.

2.2.1 Constraint Solving

Constraint solving is a well studied topic in computer science. This is a famously difficult problem to solve and no efficient computer algorithm has been discovered to solve large instances (Mohr & Henderson, 1986). In constraint solving, constraint propagation methods do additional bookkeeping in order to prune away values from domains that would lead to dead ends later (Karth & Smith, 2017).

In their paper *A Brief on Constraint Solving*, Hoffmann and Joan-Arinyo review the constraint solving domain using 2D-geometric problems as a case study (Hoffmann & Joan-Arinyo, 2005). Their work is relevant because they outline general properties that constraint solving algorithms may exhibit. Relevant for our work are the notions of stability and intentionality. With stability, a small change in constraints leads to a nearby solution. Finally, intentionality relates to finding solutions that the user is interested in. Hoffmann and Joan-Arinyo clarifies that propagation methods do not guarantee finding a solution whenever one exists.

2.2.2 Model Synthesis

Building on constraint solving algorithms, in 2009 Paul Merrell published his work on *model synthesis* (Merrell, 2009). Model synthesis is a method for procedurally generating complex yet coherent 3D-models by utilizing constraint solving techniques. His work extends previous work on texture synthesis by Efros and Leung which works for 2D (Efros & Leung, 1999). In model synthesis, modules of geometry in 3D-space adhere to constraints that specifies required symmetry in vertices. Given a set up input modules, model synthesis procedurally generates a larger 3D-model

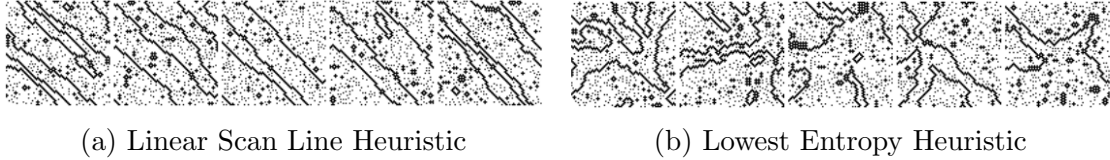


Figure 2.2: Comparison between cell selection heuristics. Source: Gumin, 2016.

by contiguously solving local constraints.

Paul Merrell’s work was popularized in 2016 when Maxim Gumin published an adaptation of model synthesis: *Wave Function Collapse* (Gumin, 2016).

WFC uses the concept of *Shannon entropy* (Bromiley et al., 2004) to choose which cell to collapse. This is a concept from information theory, where entropy defines how much is known about something. Very little is known about a cell with high entropy, whereas much is known about a cell with low entropy. WFC chooses to collapse the cell with lowest entropy.

Gumin states that the minimal entropy heuristic is mainly due to a pattern he noticed in how humans draw, which makes the algorithm more enjoyable to watch (Gumin, 2016). Merrell’s work uses a *linear scan* over the cells, which can leave noticeable directional artifacts within the generated image (Merrell, 2009). In contrast, the *lowest entropy* heuristic does not exhibit the same directional artifacts (see figure 2.2).

After a cell is collapsed, more is known about the neighboring cells, since the adjacency rules inform what tiles they can become. These constraints are propagated, so that the entropy of the surrounding cells is reduced.

WFC uses the family of constraint solving known as *Arc Consistency* (Mackworth, 1977). Each cell in the grid can be considered a node with connections to each neighbor known as arcs. Mackworth proposes a fast constraint propagation method for arc consistency known as AC-3.

The original implementation of WFC (Gumin, 2016) uses the AC-3 algorithm for the constraint propagation. Shortly thereafter, Karth and Smith, 2017 published an even more efficient implementation of the Wave Function Collapse algorithm using another constraint propagation algorithm AC-4.

AC-4 has a high start up cost due to the data structures, meaning it is not always faster than AC-3. For our use-case of generating small 2D tilemaps, we decided to implement AC-3, due to its relative simplicity and faster start up time.

2.2.3 The challenge with WFC

One of the main problems with the WFC algorithm, and many constraint solving algorithms in general, is that it may reach a state that is not completable. This is

referred to as reaching a *contradiction* (see figure 2.3). Choices made earlier during the execution of the algorithm makes it impossible to find a solution that satisfies all constraints. A few strategies have been developed to deal with this. One strategy is backtracking, where the latest operations are reversed to go back to an earlier state. Another is to simply start over with a different initial configuration (Sandhu et al., 2019).

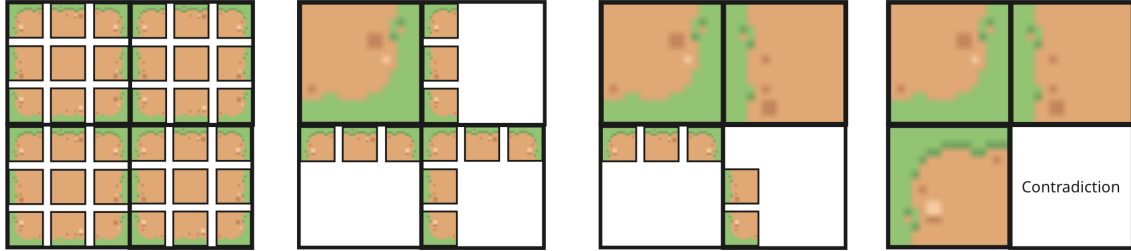


Figure 2.3: WFC resulting in a contradiction.

Another problem is that the results it generates can look very random. While adhering to local constraints (adjacency rules), it does not consider more global constraints, such as playability (i.e. the player can explore the space).

Some of the randomness of WFC can be tuned by adding weights to WFC. Whenever WFC chooses a tile for a cell to collapse to, it can choose one according to a *weighted* random function. Tiles with a higher weight are more likely to be chosen than tiles with a low weight.

Sandhu et al., 2019 implement a version of WFC that supports constraints over larger distances and non-spatial constraints. Further, it supports weighted random and even weight recalculation based on neighbors. The goal of these extensions is to make WFC better at design-level constraints.

An alternative to improve WFC is to improve the input to the algorithm. However, manually tuning the input can be challenging if the search space is large. Further, it may not be clear how to achieve a desired result from the input alone. One approach is to search through the space of possible inputs.

Brellas, 2021 uses an algorithm to optimize the input for WFC. They use an implementation of WFC (selfsame, 2016) that takes a set of tiles and a sample image as input. From the sample image, it derives a set of rules. Brellas generates input images to allow WFC to create levels with high variety, yet coherent looking results. Brellas mentions how future work could try to optimize other parts of the WFC input. Brellas highlights how the weights in particular might be a very good option to control the output of the algorithm.

Brellas uses a *genetic algorithm* to find good inputs for WFC, looking through a bunch of candidates and evaluating them. This is an example of Search-Based Procedural Content Generation.

2.3 Search-Based Procedural Content Generation

While generate-and-test methods either accepts or discards a solution, SB-PCG uses an evaluation (or fitness) function to determine *how good* a solution is. The fitness value of the previous individuals is then used to generate new individuals (Togelius et al., 2011).

Most SB-PCG is in some way evolutionary, which is also seen in the vocabulary. Some candidates will be selected during the *selection* phase. Often individuals with higher fitness will be selected, though there exists many approaches to selection. Then, the new generation is stochastically varied in the *variation* phase. The variation can be mutative, mutating only one individual, or use crossover to create a new individual from multiple parents. The new generation is then evaluated and used as a basis for the next generation.

In SB-PCG, the content is usually stored in a manner that can be expanded into the actual content seen in the game. The content that is stored is called the *genotype* while the expanded content is called the *phenotype*. One might think of the genotype as a blueprint for a building, and the phenotype as the actual building (Togelius et al., 2011).

The expansion of the genotype into a phenotype is called the *mapping* or *encoding*. This encoding can be either *direct*, where it is relatively computationally simple to map from genotype to phenotype, or *indirect*, where it may be very computationally expensive to map from genotype to phenotype (Togelius et al., 2011).

It is always desirable that a small change to the genotype leads to a small change in the phenotype and a small change to the fitness. When this property holds, the representation exhibits *high locality*. The chosen representation should also be capable of representing all interesting solutions. However, indirect encodings makes this difficult, since there could be phenotypes for which no genotypes map, and it is often not simple to detect this (Togelius et al., 2011).

Some of the concepts from SB-PCG echoes the concepts from constraint solving. Hoffmann and Joan-Arinyo, 2005 refers to stability, where a small change in the constraints, leads to small change in the output. This is similar to the idea of locality within SB-PCG. The fitness function relates to Hoffman and Joan-Arinyo’s concept of intentionality. The high fitness individuals will be more interesting to the user than low fitness ones. It therefore seems appropriate to use SB-PCG for the purposes of finding good input to constraint solving algorithms.

Brellas, 2021 uses a genetic algorithm to mutate the input image to WFC. The genetic algorithm has a population of individuals, where the best are kept between generations according to some selection method. The fitness function is based on how similar the pixels between tiles are.

Montoya et al., 2025 extends WFC by implementing a Compositional Pattern Producing Network (CPPN) to improve the ability for WFC to adhere to global constraints. This is done through user-guided evolution, where the evaluation is only done by humans.

In this thesis, we use a different approach. Our goal is to illuminate what kinds of levels a set of tiles can create. The user should be able to sample a set of weights that accurately produces levels with certain characteristics and rarely reaches contradictions. In other words, we want diverse solutions of high quality.

2.3.1 Quality-Diversity

Many PCG algorithms aim to optimize the content for quality by maximizing the fitness of a population, e.g. genetic algorithms (Mitchell, 1998, Ch. 1.6).

Some SB-PCG algorithms aim to discover as many *different* solutions as possible. Such algorithms are called *illumination algorithms* since they illuminate the space of all possible solutions (Mouret & Clune, 2015). *Novelty search* (Lehman & Stanley, 2008) is an algorithm, which rewards anything new, while not optimizing for any quality-based objective.

While only optimizing for fitness can get stuck in a local optimum, only optimizing for novelty can end up never finding anything good. Therefore, a family of algorithms called *Quality-Diversity* (QD) algorithms aim to find the balance between finding high-quality, but diverse solutions (Gravina et al., 2019).

Multi-dimensional Archive of Phenotypic Elites (MAP-Elites)

Illuminating the entire search space (i.e. by exhaustively searching through all solutions) is often infeasible with very large search spaces. Further, it is not helpful, if many of these solutions are undesirable.

Therefore, the designer needs some control of what areas of the search space are desirable to explore. One way to do this is through behavioral diversity. The designer can define certain features (behaviors) that the designer wants to see variation across. By defining multiple features, one can imagine each as an axis, and the combination of them as a multi-dimensional *behavior space*. Then, the QD algorithm can illuminate this behavior space, rather than trying to illuminate the entire search space (Gravina et al., 2019).

The *Multi-dimensional Archive of Phenotypic Elites* (MAP-Elites) algorithm is introduced by Mouret and Clune, 2015 as an illumination algorithm. It can solve *black-box optimization problems*, as requires no domain knowledge of the genotype-to-phenotype mapping.

MAP-Elites partitions the behavior space into cells in a multi-dimensional archive,

and evolves and evaluates the best individuals using their phenotypic fitness values. The product of the algorithm is a collection of high-performing individuals that are all diverse along the behavioral axes (Mouret & Clune, 2015).

Constrained MAP-Elites (CME)

The quality criteria of QD algorithms is usually either based on *local competition* or *constraints*. Local competition pits individuals that fill the same niche against each other, promoting the best performing individuals in that niche. This is used in MAP-Elites, since each individual only competes with individuals in the same cell. Constraint-based quality criteria usually use "hard constraints" to divide the population into *feasible* and *infeasible* solutions (Gravina et al., 2019).

One approach to dealing with hard constraints could be to simply discard any individuals that do not fit the constraints. However, if the process of evaluating or mutating an individual is computationally expensive, all that work is discarded and therefore wasted.

Another approach to dealing with constraints is the *Feasible-Infeasible 2-Population* (FI-2Pop) algorithm (Kimbrough et al., 2008). FI-2Pop simultaneously evolves two populations. It evolves the infeasible population towards feasibility, whereas it optimizes the feasible population for fitness. There is no direct crossover between the two populations. The idea is that infeasible solutions may become feasible through evolution. Therefore, the computational work evaluating and evolving the infeasible individual is not wasted.

A modification of MAP-Elites called *Constrained MAP-Elites* (CME) combines MAP-Elites with FI-2Pop. In each cell, CME keeps a feasible solution along with a set of infeasible candidates. Once an infeasible solution becomes feasible by satisfying all constraints, CME tests for fitness and keeps it only if it is better than the current solution in that cell. As such, CME keeps only the best feasible individual in each cell, as well as a number of candidates that could potentially become feasible and better than the current individual (Khalifa et al., 2018).

For this thesis, we want to evolve an archive of input weights to WFC that rarely reaches a contradiction. We use CME, where the hard constraint is a threshold for how often WFC has to complete. Further, we want an input to WFC to generally generate the same kind of output. Therefore, our fitness function is based on how close the levels generated by WFC are to each other in behavior space. Lastly, by using MAP-Elites we enable users to choose what aspects of the levels they are interested in seeing diversity in. This enables mixed-initiative PCG, by evolving input weights to Wave Function Collapse using Constrained MAP-Elites.

3 Methodology

In this chapter, we present and discuss the product we have developed for the purpose of investigating the research question. Firstly, we examine the structure of the technical product. Then, we elaborate on our implementation of the Wave Function Collapse algorithm, our MAP-Elites implementation and how we visualize the results from both of them.

3.1 Project Structure

Our project repository can be found at <https://github.com/OliFryser/masters-thesis/>. The user can interact with the project in a few ways. One way is to use our command line interface *Infinimon CLI* to generate an archive of input parameters for WFC. Another is to open our *Archive Explorer* tool, which lets the user see what kinds of levels can be created from the genomes in the archive. The *Archive Explorer* also shows how the Wave Function Collapse algorithm generates levels in real-time.

The application is separated into modules with separate responsibilities (see figure 3.1). The core algorithms, MAP-Elites and Wave Function Collapse, are implemented independently of each other so that they work in isolation. We will go into more details with the implementation of these algorithms in the following sections.

To bridge the gap between our core algorithm modules and our client modules, we have implemented two modules. The *Tilemap Analysis* module can take an input tilemap and analyze it into a tileset and a set of rules. We will expand on this module in section 3.4. The *Domain* module contains all the logic for the domains we use for our experiments (see section 4.1). This separates the domain-specific code from the generic Wave Function Collapse and MAP-Elites algorithms, making these modules reusable and independent from our domains.

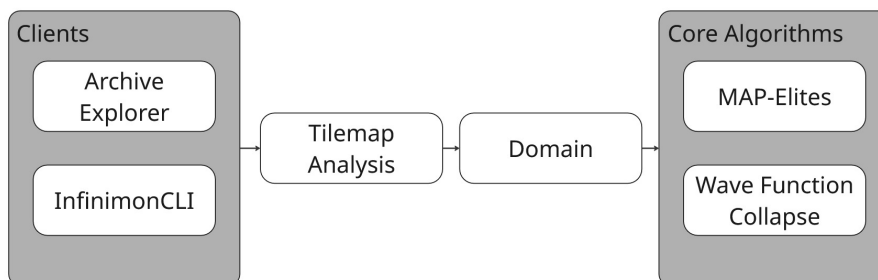


Figure 3.1: An overview of our modules and their dependencies.

3.2 Wave Function Collapse

The Wave Function Collapse (WFC) algorithm serves as the level generation algorithm case study for our research.

Our implementation uses the Simple Tiled Model. Section 2.2 outlines the general background for the algorithm. A set of tiles is provided along with a set of rules that dictate which tiles may be placed next to each other. Our implementation also accepts a set of weights that influence the probability of choosing a given tile while creating the level. This is important, since the weights are the part of the input, we are evolving. See algorithm 1 for a high level overview of the algorithm.

Algorithm 1 Wave Function Collapse

```

1: procedure WAVEFUNCTIONCOLLAPSE
2:   level  $\leftarrow$  new Level
3:   while  $\neg$ level.IsCollapsed()  $\wedge$  level.IsFeasible() do
4:     STEP(level)
5:   end while
6: end procedure

7: procedure STEP(level)
8:   cell  $\leftarrow$  PICKCELL(level)
9:   propagationStack  $\leftarrow$  new Stack
10:  COLLAPSECELL(level, cell, propagationStack)
11:  while propagationStack is not empty do
12:    cell  $\leftarrow$  propagationStack.Pop()
13:    PROPAGATE(level, cell, propagationStack)
14:  end while
15: end procedure

16: procedure PROPAGATE(level, collapsedCell, propagationStack)
17:   for each cell in level.Neighbors[collapsedCell] do
18:     if level.Collapsed[cell] then
19:       continue
20:     end if
21:     excludedOptions  $\leftarrow$  level.PruneInconsistentOptions(cell)
22:     if excludedOptions.HasAny() then
23:       level.UpdateSumOfWeights(cell, excludedOptions)
24:       level.ReduceEntropy(cell)
25:       propagationStack.Push(cell)
26:     end if
27:   end for
28: end procedure

```

The propagation stack holds which cells to propagate from. When a cell is collapsed, we choose a tile from its available tiles. If this excludes any tiles (i.e. the

cell had multiple options), the cell is put onto the propagation stack.

In the **Propagate** step, we visit all cardinal neighbors of the cell and prune any tiles from the options of the neighbor cell that are inconsistent with the options in the current cell. This ensures that all neighbors are consistent with the current cell. If any tiles were pruned this way, the neighbor cell is pushed onto the propagation stack.

3.2.1 Data Oriented Design

Runtime performance is important for this application, as the more runs we can make, the larger search space we can cover.

Our WFC implementation follows principles from Data-Oriented Design (Acton, 2014). Specifically, we use the structure-of-arrays design pattern, where each piece of information related to tiles or cells is split into separate arrays with length equal to the number of tiles or cells, respectively (see figure 3.2).

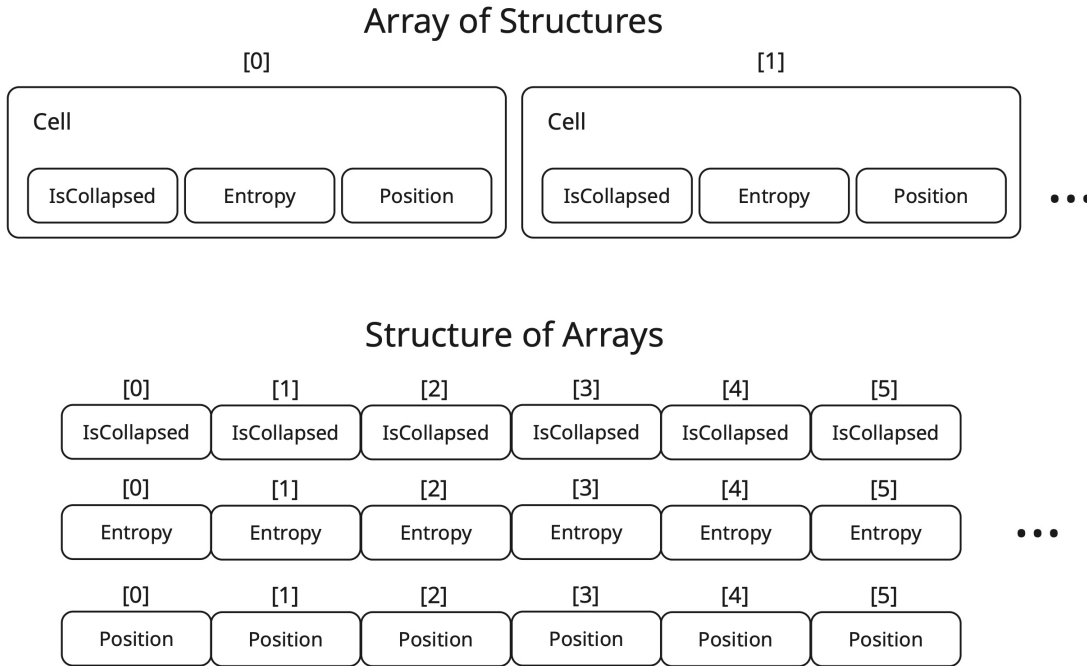


Figure 3.2: Memory layout of array of structures vs structure of arrays.

Whenever data is read from the main memory, the CPU fetches an entire *cache line*, typically 64 bytes. This means that even if only a single integer (4 bytes) was requested, the following 60 bytes will be put into the much faster cache. If the data we request is in the cache, this speeds up the program tremendously (Bryant & O’Hallaron, 2016).

We can iterate over all values of a certain type very quickly, using the structure-of-arrays data layout (Acton, 2014). In WFC, every step scans every cell of the level

to find the cell with the lowest entropy. This scan is much faster than using a single array to hold all entropy values than packing it into an array of `Cell` structs.

3.2.2 BitArray

After a benchmark analysis, we discovered that our memory consumption increased greatly as we tried to generate larger levels. The culprit was the propagation step, where many set operations are performed such as union and intersection operations to construct valid sets of neighbor tiles.

To solve this problem we have replaced *HashSets* with *BitArrays* (Microsoft, n.d.) to store the valid neighbor tiles of a cell. So instead of storing the individual tile indices as separate 64-bit integer values, we store a bit-string where the i th bit represents whether or not tile i is valid next to the cell. This implementation saves a lot of memory as the set operations no longer need to allocate new arrays to store the new sets. Instead, all the bits are modified in place, meaning no new allocations. Further, the *BitArray* class takes advantage of hardware accelerated bit-operations such as *AND*, *OR* and *XOR* which we use to perform the set operations intersection, union and difference, respectively.

We did a comparative benchmark-analysis that revealed a 2-3x performance increase and 100x memory usage reduction for large levels as seen in table 3.1.

Table 3.1: Benchmark results comparing WFC implementations using *HashSet* vs. *BitArray* for set operations.

Method	Dimension	Mean (s)	StdDev (s)	Median (s)	Allocated
HashSet	10	0.0196	0.0110	0.0186	118.14 MB
BitArray	10	0.0056	0.0029	0.0057	4.83 MB
HashSet	50	1.6335	1.4970	1.2820	175.61 MB
BitArray	50	0.4591	0.3790	0.3827	8.31 MB
HashSet	100	5.0053	3.7566	3.8990	24041.51 MB
BitArray	100	1.5148	1.0840	1.2008	223.46 MB

3.3 MAP-Elites

As mentioned in 2.3.1, MAP-Elites is a black-box illumination algorithm. It does not need to know the inner workings of the genotype-to-phenotype mapping to optimize and illuminate the behavior space.

We treat WFC as a black-box algorithm for the purposes of integrating it with MAP-Elites. Thus MAP-Elites can vary only the input, and evaluate the results only with the output of WFC.

We want to generate an archive of input to WFC, such that it runs into fewer dead-ends and becomes more consistent in the types of levels it creates. Thus, WFC works as the genotype-to-phenotype mapping between the genotype (the input to WFC) and the phenotypes (the generated levels).

As explained in section 2.2.3, the weights define how likely each tile is to be placed in the level. Early in the project, we saw that by manually setting the weight of one tile very high, that tile was consistently represented in the generated levels. This indicated that the weights were a good candidate for mutation. Further, Brellas, 2021 mentions that the weights could be a good candidate for mutation in further work.

Therefore, our genotype consists of a set of weights $W = \{w_1, w_2, \dots, w_N\}$, where a weight w_i represents the weight of the tile t_i , and N is the number of unique tiles. Each weight is in the range $[0; 1]$.

The rules and tiles available for WFC are not part of the genome and thus remain constant for all genomes in the archive. It is worth noting that a weight of 0 for a tile is equivalent to removing that tile entirely, since the odds of picking that tile are 0.

In our implementation, the user specifies G initialization iterations and I variation iterations. For G iterations, MAP-Elites initializes new individuals randomly. For each individual it generates, the algorithm evaluates the individual to obtain its fitness and behavior. The algorithm finds the correct cell in the archive and places the individual in that cell if the cell is empty or if the individual has a higher fitness than the current entry in that cell.

Then, for the next I iterations, the algorithm samples an individual from the archive and randomly varies it. The algorithm either saves the individual or discards it, according to the same rules as before.

3.3.1 Constraining MAP-Elites

Constrained MAP-Elites (CME) combines the FI2-Pop algorithm with MAP-Elites to allow MAP-Elites to maintain a population of feasible and infeasible individuals. The feasible population is ranked on fitness, whereas the infeasible population is ranked on feasibility.

In our project, the feasibility criterion is defined as the percentage of genotype-to-phenotype mappings that do not reach a contradiction in WFC. Thus, a feasible individual is an individual that finishes more often than some threshold specified by the user. This addresses our *reliability* criterion of WFC from our research questions. Only when an individual is feasible do we consider the fitness of the individual. The fitness addresses the *consistency* criterion of WFC. Later in this

chapter, we elaborate on how the fitness and feasibility is calculated.

Although FI2-Pop can hold any amount of individuals in the population, in our implementation each cell holds one *feasible* and one *infeasible* individual. We made this decision for the sake of simplicity in analysis and implementation of the algorithm.

The user supplies a probability that the feasible individual is chosen over the infeasible one when a cell is sampled in the archive. If a feasible individual is chosen more often, more resources will be spent optimizing for fitness. If an infeasible individual is chosen more often, more resources are spent optimizing for feasibility.

We will use CME for all our experiments since it addresses both the reliability and consistency criteria of WFC. See algorithm 2 for an overview of the algorithm.

3.3.2 Generic Implementation

We implement MAP-Elites generically to make it reusable for most optimization purposes. We define an interface called `IIndividualHandler`. The individual handler supplies all the methods for how to handle an individual in MAP-Elites. An individual is a genome in the archive. The methods required for MAP-Elites are: `CreateRandom`, `Mutate`, `Evaluate`, and `GetKey`.

For our project, we implement an individual handler which uses WFC in the evaluation. This individual handler is in the `Domain` project. The individual handler takes a domain as input and does the required setup for that domain.

3.3.3 Evolutionary Strategies

There are many approaches to evolving genomes. The goal is to choose an approach that efficiently and effectively explores the search space. It is desirable that it *converges* fast and with good results. When it converges, we expect that spending more time on computation does not yield better results.

Much research has already been done in evolving a series of floating point numbers, exactly like the weights in the genome. In this section, we present some standard approaches and discuss how we adapt these to our project.

Ha defines an evolutionary strategy as "an algorithm that provides the user with a set of candidate solutions to evaluate a problem" (Ha, 2017). The evaluation uses an objective function that yields a fitness value.

For our purposes, MAP-Elites uses the evolutionary strategy when it randomly varies an individual at each iteration and then uses our fitness function to evaluate it. How Ha presents these evolutionary strategies does not provide a one-to-one mapping to our use-case. Each evolutionary strategy has to be adapted to work within MAP-Elites.

Algorithm 2 Constrained MAP-Elites with Early Convergence. Refer to table 4.1 for explanation of G, I and C.

```
1: procedure MAPELITES(G, I, C)
2:   archive  $\leftarrow \{\}$ 
3:   for i  $\leftarrow 0$  to G do
4:     individual  $\leftarrow$  CREATERANDOM
5:     EVALUATEANDSAVE(individual, archive)
6:   end for

7:   iterationsSinceLastSave  $\leftarrow 0$ 
8:   for i  $\leftarrow 0$  to I do
9:     if iterationsSinceLastSave  $\geq C$  then
10:      return archive  $\triangleright$  Early termination due to convergence
11:    end if
12:    individual  $\leftarrow$  SAMPLE(archive)
13:    newIndividual  $\leftarrow$  MUTATE(individual)
14:    if EVALUATEANDSAVE(newIndividual, archive) then
15:      iterationsSinceLastSave  $\leftarrow 0$ 
16:    else
17:      iterationsSinceLastSave  $\leftarrow$  iterationsSinceLastSave + 1
18:    end if
19:  end for

20:  return archive
21: end procedure

22: procedure EVALUATEANDSAVE(individual, archive)
23:   newEntry  $\leftarrow$  EVALUATE(individual)
24:   key  $\leftarrow$  GETKEY(newEntry)
25:   current  $\leftarrow$  archive[key]
26:   if current = null then
27:     archive[key]  $\leftarrow$  newEntry
28:     return true
29:   end if
30:   if newEntry.IsFeasible  $\wedge$  current.Fitness < newEntry.Fitness then
31:     archive[key]  $\leftarrow$  newEntry
32:     return true
33:   end if
34:   if current.Feasibility < newEntry.Feasibility then
35:     archive[key]  $\leftarrow$  newEntry
36:     return true
37:   end if
38:   return false
39: end procedure
```

Ha presents a few evolutionary strategies. One of them is the "simple evolution strategy". The idea is to use a normal distribution to sample the solutions using a *mean* solution (μ) and a standard deviation (σ). It discards all solutions except the fittest and uses that as the new mean (Ha, 2017).

An issue with this approach is that the standard deviation is fixed. The standard deviation determines how far the algorithm explores in each iteration. A large standard deviation explores widely in the search space, but may overlook some good nearby solutions, since it steps too far. By "nearby", we refer to the distance in search space from the mean. CabbageCat, 2022 highlights that in the beginning we may want a large standard deviation to find many different candidates, but already good solutions may only need a few small adjustments to become great.

Ha, 2017 mentions another evolutionary strategy called *Covariance-Matrix Adaptation Evolutionary Strategy* (CMA-ES). The idea is to adapt the standard deviation by considering how far good solutions are from each other. If good solutions are far from each other, more of the search space should be explored to find the best solutions. Thus, our standard deviation should be larger. If the good solutions are clustered together, the standard deviation should be small, so nearby solutions are explored. By creating a *Covariance-Matrix*, the CMA-ES can change the shape and size of the standard deviation, such that it explores in the direction of good solutions with a step size that makes sense.

To adapt this to MAP-Elites, Fontaine et al., 2020 introduce the *CMA MAP-Elites* (CMA-ME) algorithm. CMA-ME initializes a population of *emitters*. An emitter is initialized with a *mean genome* sampled from the archive. The emitter can then mutate a new genome from that mean genome. After generating some amount of genomes, the CMA state then adapts its covariance-matrix to better mutate the next individuals by setting the mean genome to a new mean among the best solutions it generates.

Fontaine et al., 2020 present three kinds of emitters: Optimization emitters, random-direction emitters, and improvement emitters. An optimization emitter ranks its population by how much it improves the fitness over the mean genome. The random-direction emitter picks a random direction in behavior space and then ranks the population based on how far along that direction the new genome has moved relative to the mean genome. Improvement emitters rank a new genome by how much it has improved compared to the cell it lands on in the archive. Fontaine et al. highlight how this strikes a balance between exploration and optimization by greatly rewarding hitting empty cells.

To adapt CMA-ME to constrained MAP-Elites, we propose a new emitter: *The feasibility emitter*. The feasibility emitter ranks the population by how much the new genome improves the feasibility compared to the mean genome. If it crosses

the feasibility threshold it is rewarded greatly. We also modify the optimization emitter to punish a genome greatly if it becomes infeasible. The rationale for this is that the genome should not be rewarded for getting fitter, if it does so at the cost of becoming infeasible. We name this adaptation of CMA-ME "Covariance-Matrix Adaption for Constrained MAP-Elites" or CMA-CME for short.

For our implementation, we implement only the optimization emitters, random-direction emitters and feasibility emitters. We leave the implementation of improvement emitters for CMA-CME for future work.

In chapter 4, we compare how CME performs with a fixed standard deviation compared to CMA-CME with emitters.

3.3.4 Random Initialization

The initialization of the archive should be done in a manner that fills the archive as much as possible. Each weight is uniformly sampled in the range $[0; 1]$. Since it is uniformly random sampled, this should give a good initial coverage to start randomly varying from.

Fontaine et al., 2020 highlight that collapsing a high dimensional search space into a low-dimensional behavior space has a distorting effect on the types of individuals that can be uniformly sampled.

We also distort the search space, which is N -dimensional, by collapsing it into a two-dimensional behavior space. Therefore, we expect that the initialization will only fill a few cells in the archive, since we sample them uniformly in search space. This gets worse the more dimensions the search space has, i.e. the more tile types are in the tileset.

3.3.5 Random Variation

When the archive is initialized, the algorithm randomly samples an individual at each iteration and randomly varies it. As mentioned in section 3.3.3, there are many ways to do this random variation. Usually, it falls into either of two categories: *Mutation* or *crossover*. We use relative weights between 0 and 1, so two genomes might have different relative differences between their weights, and therefore crossing them does not seem relevant. Instead, we focus exclusively on mutation. See table 3.2 for an overview of our different mutation strategies.

Our normal distribution sampling is done according to the formula $\mathcal{N}(\mu, \sigma)$, where μ is the weights of the sampled individuals and σ is the standard deviation. The standard deviation is a *hyperparameter*. In section 4.2, we run experiments on different standard deviation values to find a suitable value for the rest of the experiments.

Table 3.2: Overview of different mutation strategies.

Mutation	Sampling	Genome Changes	Std. Deviation
All Tiles	Normal Distribution	Every weight	Fixed
1/3 Tiles	Normal Distribution	1/3 of weights	Fixed
Special Tile	Normal Distribution	Only special tile	Fixed
CMA-CME	Covariance-Matrix	Every weight	Adaptive

All Tiles mutates all the weights in the archive according to the normal distribution. This makes each iteration more volatile since the genome moves somewhere new in each direction in the search space. However, using the normal distribution, most steps will be small and some will be larger.

1/3 Tiles mutates a third of the weights chosen randomly. We expect this method to be less volatile since it takes more time to finish. However, each iteration also explores less, which could lead to less coverage and slower convergence.

Special Tile mutates only the weight of the *special tile*. More on that in the following section.

CMA-CME uses our CMA-CME implementation. We expect that the archive explores more of the archive using this approach and that it converges faster since it adapts the standard deviation and exploration direction in search space.

3.3.6 Evaluation

MAP-Elites evaluates the individual on its phenotype. However, WFC is a stochastic algorithm, so it can generate many phenotypes from the same genotype. Therefore, we average the level characteristics over M iterations of WFC. A genotype therefore maps to an average of a collection of M phenotypes.

Larger values of M should improve how accurately a genotype reflects its archive phenotype, but increase evaluation time. Users must balance this trade-off to ensure both accuracy and a reasonable runtime. In section 4.4, we present results using different values of M .

Behavior

The behaviors define what kind of diversity the user wants to explore. An almost infinite number of behaviors could be used, as long as the user implements a way to extract them from the individual.

The first behavior we use for our experiment is the *special tile %*. The special

tile can be any tile in the tileset. In our Pokémon tileset, it is the red flower tile. This simulates a use-case where the user wants to see how Pokémon levels look with many flowers as opposed to few flowers. We expect that this behavior correlates directly to the weight of the special tile, making it easier to find through mutation of weights.

Another interesting behavior to examine is the variation in the generated level. For instance, it may be desirable to sample a level with vast spaces of grass fields and flowers, while at other times chaotic levels with many different tiles could be appropriate. This behavior also does not translate directly into tuning the weight of a specific tile, as it likely requires balancing the weights to get many different tiles. Therefore, we expect it to be more difficult for CME to fill this axis of behavior space. We dub this behavior *variation %*.

We can calculate the *variation %* in two ways. The first is through the *unique count* implementation. The variation is calculated by counting the number of unique tiles in the generated level and dividing by the total number of tiles to choose from. The shortcoming of this approach is illustrated by a level where every tile is used only once, except one tile which is used everywhere else. According to the calculation it has the max variation, however, in our opinion it does not look very varied at all.

To bridge this gap, we introduce a different way to calculate the *variation %* by using Shannon entropy H which better reflects whether the level is dominated by fewer specific tiles.

$$H = - \sum p(x) \log p(x)$$

It also worth noting that these behaviors are dependent upon each other. Gravina et al., 2019 highlights how two behavioral characteristics that are dependent on each other makes some areas of the behavior space inaccessible.

So is the case for our behavioral characteristics. A high frequency of special tiles causes less variation. Likewise, lots of variation makes it harder to have a high frequency of special tiles. We expect that filling some corners of our behavior space will be difficult, if not impossible.

Formally, we define our behavior b of a phenotype as the set of behaviors $\{t, v\}$ where t is the *special tile %*, and v is the *variation %*. We define the behaviors for M levels generated by WFC as $B = \{b_1, b_2, \dots, b_M\}$. The behavior of an individual is the mean behavior of all M levels, denoted as $\bar{b}$.

We only use the levels that complete as a basis for the phenotypic behavior. The levels that do not complete may not accurately represent the kinds of levels the genome can create. If the generation fails already after the first tile is placed, the *variation %* of that level will be very low. This skews the results by having extreme outliers.

Feasibility

An individual is first measured on its feasibility. This determines whether the individual goes in the feasible or infeasible slot of the archive.

For our implementation, the user supplies a *feasibility threshold*. Any individual with a lower feasibility than the threshold will be deemed infeasible. We measure the feasibility of an individual as the completion rate of M WFC runs. With a feasibility threshold of 0.75, an individual is deemed infeasible if their completion rate falls below 75 %.

We do this, since we believe that it is more important that a level completes rather than being of high quality. This addresses the *reliability* of WFC.

Fitness

A feasible individual is judged according to its fitness. This enables local competition within its niche, meaning only the most fit individual that exhibits certain behavioral characteristics will be kept.

For our application, one challenge is the stochastic nature of our evaluation algorithm WFC. Therefore, we want to reward consistency in our genotypes. An individual that generates phenotypes that are close in behavior space is more valuable than an individual that generates wildly distant phenotypes that just happen to average to a certain behavioral cell. Figure 3.3 shows a high fitness and low fitness individual in behavior space.

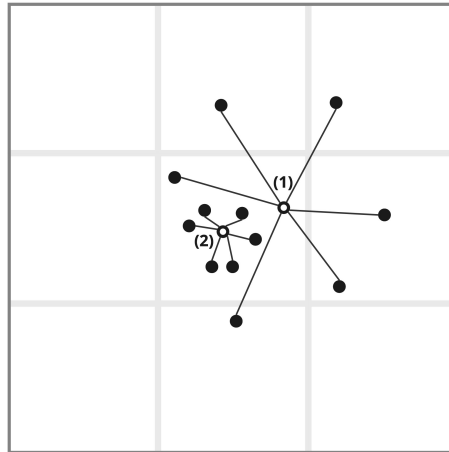


Figure 3.3: Black dots are behaviors and white dots are the average of the black dots connected to it. (1) has a large distance from the mean giving it a bad fitness score. (2) has a smaller distance from the mean giving it a better fitness in the same cell. Therefore (2) replaces (1) in the archive.

We calculate how far the behavior of a genotype is from the average of the genotype by calculating the euclidean distance from its behavior to the average. Let

$\bar{b} = \{\bar{t}, \bar{v}\}$ define the mean behavior of the genotype. Then the euclidean distance is computed as:

$$D(b_i, \bar{b}) = \sqrt{(t_i - \bar{t})^2 + (v_i - \bar{v})^2}$$

We calculate the mean distance of a genotype as:

$$meanDist(B, \bar{b}) = \frac{1}{M} \sum_{i=1}^M D(b_i, \bar{b})$$

The mean distance is near 0 when the generated levels are close, and larger when the generated levels are farther apart in behavior space. This is the inverse of what we want, since we want a high fitness when the distances are small. Therefore, the mean distance is put through an activation function that yields a high value when the distance is small and a low value when the distance is large. The fitness function F is thus:

$$F(B, \bar{b}) = e^{-meanDist(B, \bar{b}) \cdot k}$$

where k is the smoothing factor. See figure 3.4 for examples of the activation function with different smoothing factors. The higher the k , the more extreme the slope, meaning values close to 0 are spread further apart.

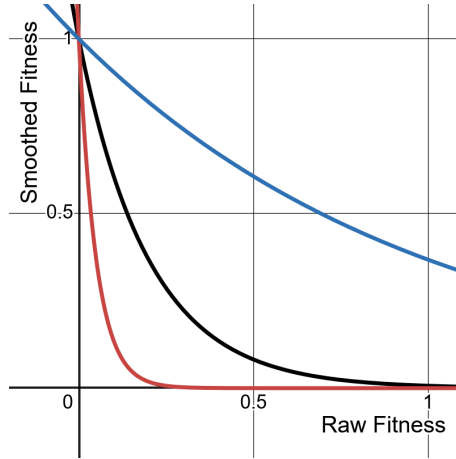


Figure 3.4: Different smoothing factors of fitness activation function e^{-kx} . $k = 1$ (blue), $k = 5$ (black) and $k = 20$ (red). Created with Desmos Graphing Calculator (Desmos, n.d.).

Since the local competition is based only on what value is larger, the choice of k has no impact on the result. It does, however, impact the readability of fitness values.

3.3.7 Key in the archive

Since the behavior axes are continuous, there is a need for some way to discretize them. The choice of discretization controls how many cells are along that axis in the archive. It also controls how fine-grained that behavioral axis will be. If a behavioral axis is continuous from 0 to 1, then dividing it into 10 cells will put all individuals with a behavior in the range $[0; 0.1)$ in the same cell, all individuals in the range $[0.1; 0.2)$ in the same cell, and so on.

Sometimes, however, it may not be that interesting to see the entire behavior range. We implement a *key ceiling* as a parameter, where the user can specify a ceiling to a behavioral axis. If the user is not interested in a *special tile %* over e.g. 20, the user can specify a key ceiling of 0.2 along that axis. This effectively shrinks the size of the cells, making the archive more fine-grained along that axis, at the cost of creating one much larger cells at the end. The last cell then covers anything above the second-to-last cell all the way up to 1. This also enables us to curb some of the unreachable sections of the archive due to the interdependency of the behavioral axes.

3.3.8 Archive Explorer

The *Archive Explorer* is a program that we have developed to interact with generated archives with a graphical user interface (see figure 3.5). It supports any archive generated with either of our domains. The user selects one of our predefined archives to browse, or the user can upload their own archive JSON file produced with *InfinimonCLI*. With an archive selected, the user can browse the different entries in the archive by clicking the cells in the behavior space. The user can click "Sample" to generate levels with that genome.

Throughout the development of this project, the *Archive Explorer* has been a valuable tool to examine the kinds of genomes MAP-Elites generated. We also believe that it is the most straightforward way to explore the results in a more interactive way. The program also displays properties about the selected archive entry, such as the *special tile %*, *variation %*, *feasibility %*, and its *fitness* score.

Secondly, the tool has been valuable as a debugging tool and in verifying the behavior of our algorithms. With it we have seen WFC step through the creation of a level, placing tile by tile.



Figure 3.5: Screenshot from the *Archive Explorer*.

3.4 Tilemap Analysis

We have implemented a *Tilemap Analysis* tool that can extract the set of tiles used in the tilemap. The tool is also responsible for determining which tiles may be placed next to each other. We have implemented two techniques to decide which tiles may be adjacent. Firstly, tiles that appear next to each other in the input tilemap may be adjacent. Secondly, the tool can analyze tile adjacency using a socket system.

Two tiles are defined to have a matching socket if the pixels along the border of one tile are identical to the pixels along the opposite border of the other tile. This method was inspired by Oskar Stålberg’s socket system used in the implementation of Townscaper (Donald, 2020).

3.5 Statistics

As stated by Mouret and Clune, 2015, ”there are many ways to quantify the quality of illumination algorithms and optimization algorithms.” The metrics provided by Mouret and Clune are primarily aimed at cross-algorithmic comparison. We use the same algorithms for all experiments, changing only the parameters, domains, or techniques within these algorithms. Therefore we adapt the metrics introduced by Mouret and Clune, 2015 to the following metrics:

Global performance: The maximum fitness of any individual in the archive. This metric reflects our search algorithm’s ability to find any good solution.

Quality-Diversity Score (QD-Score): The average fitness reached across all cells in the archive. Empty cells contribute with 0 to the average. This metric increases as coverage increases, and also increases as existing solutions are improved.

Precision: The average fitness calculated only on the populated cells of the archive. Some areas of the behavior space may be unreachable. As such, the precision can offer a more accurate estimation of the average fitness of all *reachable* cells, given that the algorithm finds all reachable cells.

Coverage: The number of populated cells divided by the total number of cells. A higher coverage means MAP-Elites illuminates more of the search space. Only cells with a feasible solution are considered *covered*.

All of the following plots have been created using Python scripts with the package *matplotlib* (The Matplotlib Development Team, 2026).

4 Experiments

In this chapter, we present our experiments. We start by presenting a hyperparameter experiment for the standard deviation when mutating the genomes. We then examine how MAP-Elites handles domains of varying difficulty.

We then examine our most complex domain in more detail. We experiment with different evaluation iterations, variation behaviors, and mutation strategies.

Each of the following experiments requires some parameters to run. Table 4.1 shows an overview of these parameters.

Table 4.1: Overview of the parameters used for our experiments.

Parameter Name	Symbol	Description
Evaluation Iterations	M	Number of levels generated for each genome.
Initialization Iterations	G	Iterations where new genomes are randomly initialized.
Mutation Iterations	I	Iterations where a new genome is generated from an existing genome via mutation.
Convergence Threshold	C	If no individual was added to the archive during the last C iterations, the algorithm terminates.
Level Dimension	D	The size of the generated levels used for evaluation. D denotes a $D \times D$ level.
Feasibility threshold	τ	Any individual with a feasibility $< \tau$ is infeasible.
Cells per axis	k	Controls how many cells each axis should have in the archive. The archive has size k^2 .
Key ceilings	t_{max}, v_{max}	The ceilings for the <i>special tile %</i> behavior (t) and <i>variation %</i> behavior (v).
Standard Deviation	σ	Controls how much a genome is mutated in a mutation iteration.

4.1 Domains

We have hand-crafted two example domains to verify that our algorithms work as expected and to highlight how our implementation of MAP-Elites works. In addition, we have included a *Pokémon Fire Red* (Game Freak, 2004) tileset to evaluate how our method handles tiles and rules which we did not create. Analyzing how CME performs on this tilemap will better reflect how the algorithm performs in real-world scenarios.

4.1.1 Letter Domain

Our first domain is the simplest. It consists of five black tiles with white letters (see figure 4.1). All tiles can go next to each other and itself. This property makes it impossible for WFC to reach a contradiction, since no tile has any constraints on what tiles can go next to it.

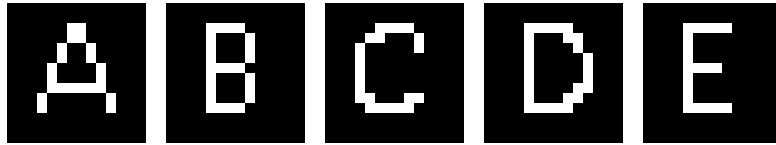


Figure 4.1: A tileset of 5 letter-tiles that will always produce a feasible solution.

With this domain, we remove the noise of contradictions to see how the algorithms handle the 'friendliest' possible input. See figure 4.2 for examples of levels that can be generated with this domain. We arbitrarily choose the A-tile as the *special tile* for this domain.

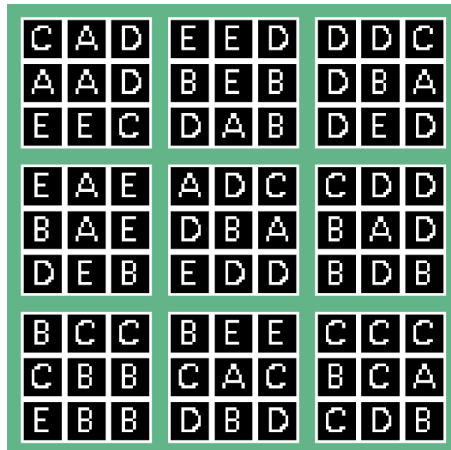


Figure 4.2: 9 example levels generated by WFC using the Letter domain.

4.1.2 Arrow Domain

This domain introduces a set of 5 tiles as seen in figure 4.3. We use the socket system to determine compatibility, which means that only tiles that have a similar border can be placed adjacently. The tileset includes an X-tile that is compatible with any arrow-tile as long as is placed along the black border of the arrow-tile. The arrows indicate where they point relative to the X-tile.

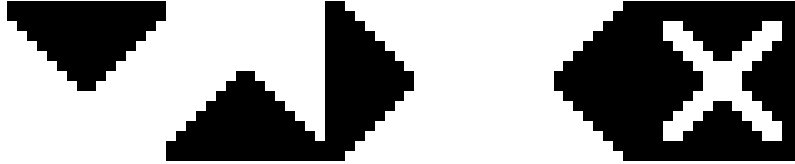


Figure 4.3: A tileset of 4 arrow-tiles and an X-tile.

See figure 4.4 for examples of levels that can be generated with this domain. The *special tile* for this domain is the X-tile. The X-tile is the only tile compatible with all other tiles.

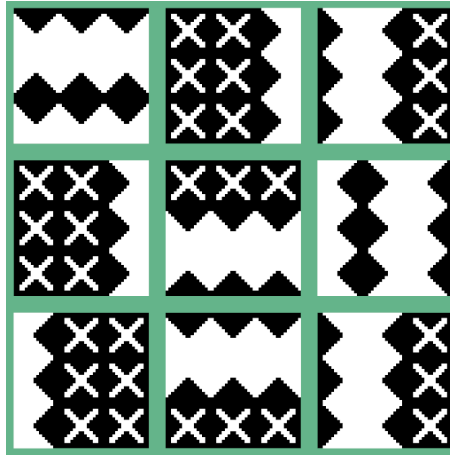


Figure 4.4: 9 example levels generated by WFC using the Arrow domain.

This domain introduces local constraints, since the arrows have to be connected to the X-tile or their own reflection. This limits the kinds of levels that can be generated by WFC. It is therefore a more challenging input to CME.

4.1.3 Pokémon Domain

As an example of real-world input, we use *Pokémon Fire Red* (Game Freak, 2004). In particular, we use the tilemap from *Pallet Town*, the starting town. It can be seen in figure 4.5.

This domain has 97 unique tiles and 1592 adjacency rules. The rules are extracted from the input tilemap using both the socket system and input tilemap adjacencies as explained in 3.4.



Figure 4.5: Tilemap of Pallet Town from *Pokémon Fire Red* (Game Freak, 2004)

We wish to see how our program scales to larger input tilemaps such as this one, as it more closely resembles what a game designer may use in a game production environment. This better reflects whether QD can improve the reliability and consistency of the WFC algorithm for real world examples. This also functions as the hardest domain for CME, since there are many more weights and constraints than in either of the other two domains. Figure 4.6 shows examples of levels generated with this tileset.

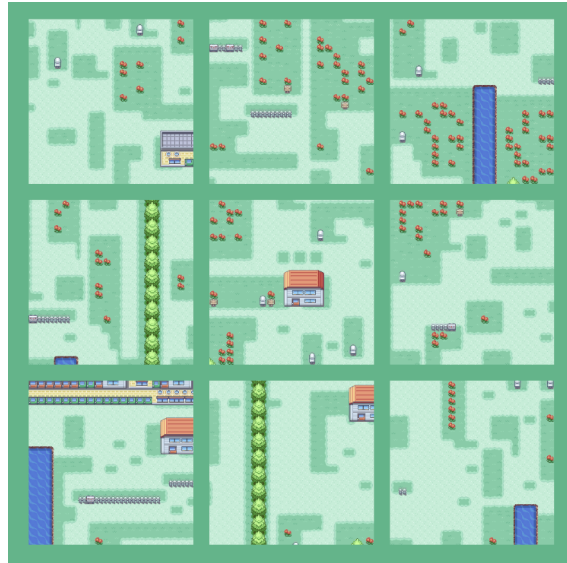


Figure 4.6: Example levels generated by WFC using the Pokémon domain.

This tilemap in particular is interesting because WFC may reach a contradiction resulting in an infeasible level. In the center of figure 4.7, WFC has placed a corner water tile close to a corner grass patch tile. This makes it impossible for the algorithm to complete the level as no series of tiles can connect the two corner tiles.



Figure 4.7: The construction of an infeasible level. Purple tiles are used to show a contradiction.

4.2 Standard Deviation

We use a hyperparameter as the fixed *standard deviation* (σ) for the mutation strategies 1-3 (see table 3.2). To determine a value for σ that leads to fast convergence and good results, we run 20 experiments with different values for σ for each of our domains.

We determine that the relevant range for sigma is $[0; 1]$, since this is the range for each of the N-dimensions of the search space. The values for σ are distributed equally throughout the range. Thus, the values tested are $0, 0.05, 0.1, \dots, 0.95, 1.0$.

We use mutation strategy 1 for these experiments. We set $D = 10$, $M = 50$, $G = 250$, $I = 5000$, $C = 500$, $\tau = 0.75$ and $k = 5$. The key ceilings differ depending on which domain we test. We use the *unique count* implementation of *variation %* for these experiments.

To judge whether a sigma value is good or bad, we compare a metric we call *global reliability*. The global reliability tracks the average fitness across all cells in the archive, divided by the global performance of the archive. This metric is similar to the *QD score* (see section 3.5), except that we divide it by the global performance. This was due to confusion on our part about the *global reliability* introduced by Mouret and Clune, 2015. We later refined this metric to the QD score, but since these sigma tuning experiments are slow, we did not have time to re-run these experiments.

Nevertheless, the global reliability does indicate both how much of the archive is filled and how well the individuals are in the archive on average. These are both desirable traits of our algorithm. However, by scaling by global performance, we discard any information about the absolute performance of the individuals in the archives.

If two σ values yield the same global reliability score, we determine which is better based on its convergence rate. Thus, we choose σ that yields the best results and use speed as a tie-breaker.

4.2.1 Letter Domain

This domain uses key ceilings $t_{max} = 1.0$ and $v_{max} = 1.0$. We expect to see a normal distribution, in the sense that nearby σ values yield a similar global reliability. Figure 4.8 shows the results of the experiment.

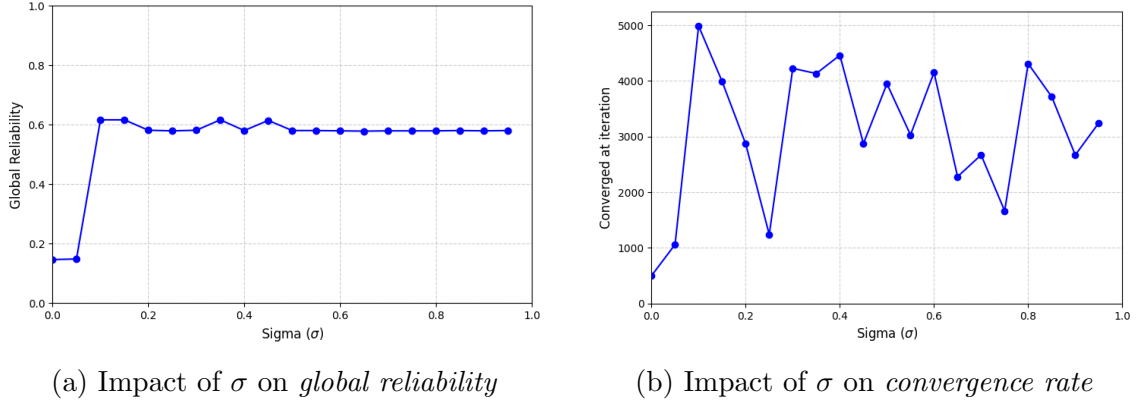


Figure 4.8: Impact of σ on *global reliability* and *convergence rate* for the Letter domain

The global reliability results surprise us. They show that σ values less than 0.1 have a much worse global reliability than all other σ values. However, all other values of sigma produce roughly the same values for global reliability. This indicates that they all find the same kinds of solutions within the given I iterations. Therefore, we consult the convergence rate. All experiments stop early because the archive has received no new candidates for C iterations (remember, the maximum number of iterations is $G + I = 5250$). Therefore, we choose a sigma that converges fast. In this case, a good candidate would be $\sigma = 0.25$, since it converges the fastest.

4.2.2 Arrow Domain

This domain uses key ceilings $t_{max} = 1.0$, and $v_{max} = 1.0$. Figure 4.9 shows the results for the this domain.

The global reliability looks similar to that of the Letters domain, although notably the global reliability is roughly halved on most results. We will discuss why this is the case in section 4.3. The global reliability is roughly the same for most results. One outlier is $\sigma = 0$. This makes sense because no mutation occurs and so we are stuck with what is made during the initialization. There is also a notable bump from $\sigma = 0.1$ to $\sigma = 0.35$

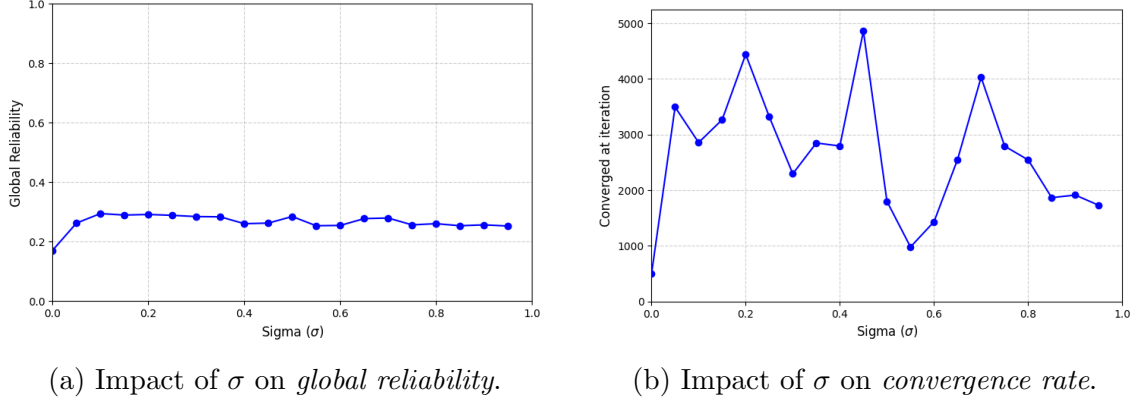


Figure 4.9: Impact of σ on *global reliability* and *convergence rate* for the Arrow domain.

Again, all experiments stop early. For this domain we choose $\sigma = 0.3$ as it has a high global reliability score and converges fast.

4.2.3 Pokémon Domain

This domain uses key ceilings $t_{max} = 0.25$, and $v_{max} = 1.0$. The reason for the low ceiling on the *special tile %* is that the algorithm has a hard time producing levels with more than 25% flowers. Further, it gives more granularity in the lower cells. Lastly, the ceiling circumvents some of the dependency between behaviors. Figure 4.10 shows the results for this domain.

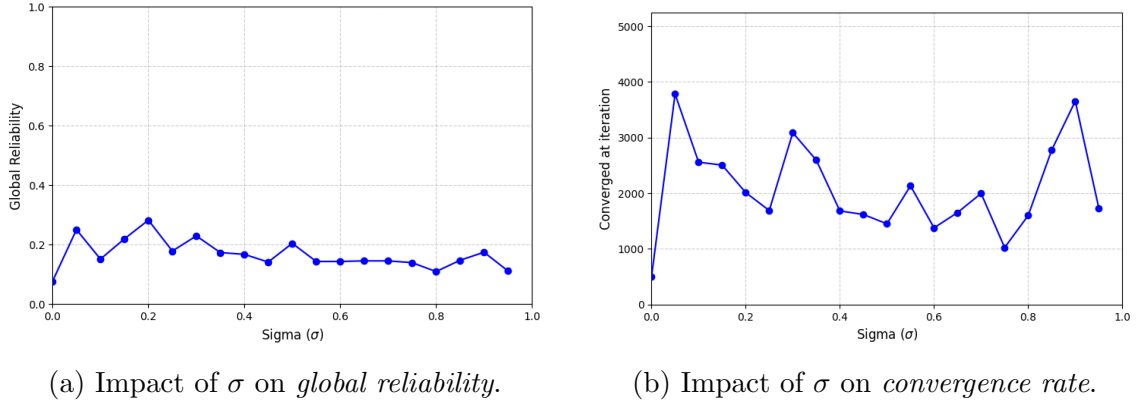


Figure 4.10: Impact of σ on *global reliability* and *convergence rate* for the Pokémon domain.

The global reliability results are flat once again, but there are some spikes. Specifically $\sigma = 0.05$ and $\sigma = 0.2$ score higher on global reliability. Consulting the convergence rate chart, $\sigma = 0.2$ converges at around 2000 iterations, whereas $\sigma = 0.05$ converges after a little under 4000 iterations. $\sigma = 0.2$ is therefore around twice as efficient. We choose this as the σ for the Pokémon domain.

4.3 Domain Difficulty

In this section, we present results from a run of CME on our three domains. For these experiments, we use mutation *All Tiles*. We set $D = 20$, $M = 10$, $G = 250$, $I = 5000$, $C = 500$, $\tau = 0.75$, and $k = 5$. As seen in the σ tuning experiments, all our domains converge or at least near convergence before 5000 iterations, so we choose this value for I . We set $D = 20$ to produce levels of 400 tiles, rather than $D = 10$ producing levels of 100 tiles. This is because the Pokémon tileset has 97 tiles, which would make it very difficult to reach high *variation %* in levels with only 100 tiles. The key ceilings differ depending on what domain we test. For the experiments in this section we use the *unique count* implementation of *variation %*. The σ values are as described in section 4.2.

To compare the difficulty of the domains, we examine the QD scores. Further, we analyze the archives using our visualization of behavior space. The archives generated for these experiments are bundled with the attached *Archive Explorer* program and can be explored interactively.

4.3.1 Letter Domain

For the Letter domain, we set $t_{max} = 1.0$ and $v_{max} = 1.0$.

We show cells in gray-scale, where the algorithm only found infeasible genomes (darker cells being more feasible than lighter cells). For the Letter domain, no genomes are infeasible. We color cells on a gradient from dark purple to yellow for the best individuals, where the algorithm found feasible genomes. A high fitness genome is bright yellow, whereas a low fitness genome is dark purple.

The archive of the letter domain is plotted in behavior space in figure 4.11. The archive contains many genomes and consists mostly of high fitness genomes. We denote (x, y) as the coordinate of a cell in the archive. The bottom left cell is $(0, 0)$.

The empty cells in the archive follow from the domain and behavior space partitioning. Since we partition the behavior space into 5 cells, the first cell contains behaviors in the range $[0; 20)$ %, the next cell contains behaviors in the range $[20; 40)$ % etc. The tilemap contains 5 tiles. Thus, no phenotype can have a variation % less than 20 %, since one tile is 20 % of the tileset ($100/5 = 20$). This explains why no cells are filled in the first row of variation count.

The empty cells $(1, 1)$, $(2, 1)$ and $(3, 1)$ follow from the observation that all cells on $(x, 1)$ must contain only one tile. Thus, it is only the A-tile, landing in cell $(4, 1)$, or a single tile that is not the A-tile, landing in cell $(0, 1)$.

During the $G = 250$ iterations, the archive quickly stagnates in both QD score and coverage (see figure 4.12). We uniformly sample the search space in the initial-

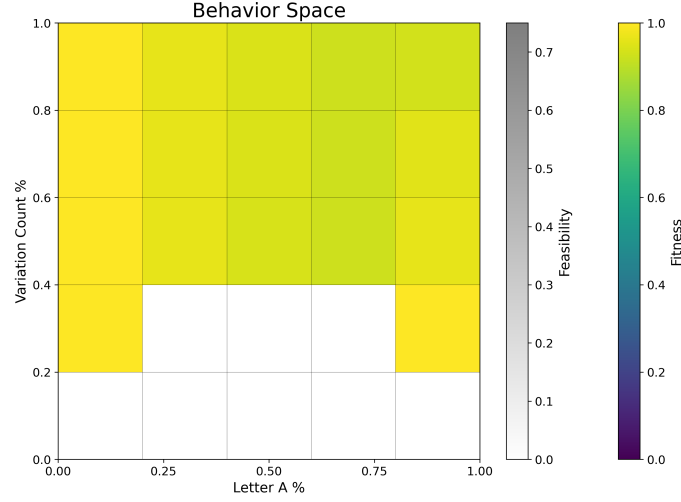


Figure 4.11: Behavior space of the archive generated for the Letter domain.

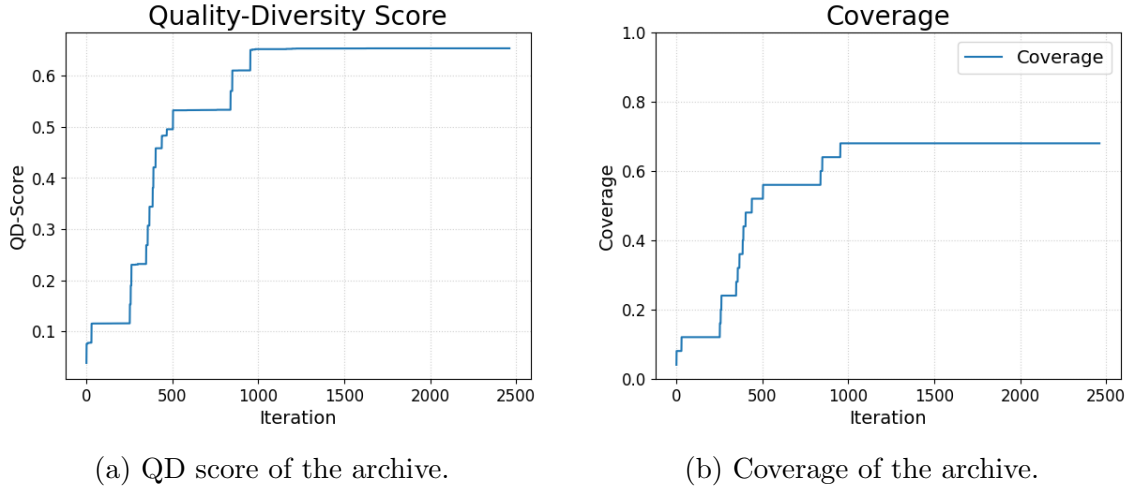


Figure 4.12: QD score and coverage of the archive for the Letter domain.

ization, but distortions introduced by mapping from search space to behavior space cause the uniform sampling to land only in the same parts of the behavior space.

After 250 iterations, the algorithm starts mutating genomes in the archive, leading to a sudden spike in the coverage. Because the algorithm is no longer sampling uniformly, but using mutation through the search space, the algorithm can illuminate new parts of the behavior space. The coverage and QD score are correlated, since the QD score metric takes the average fitness across all cells. Thus, a low coverage leads to a lot of cells with a fitness value of 0.

Because we know only 17 out of 25 cells can possibly be reached, the highest possible QD score of the letter domain is $17/25 = 0.68$. The archive has a QD score of 0.65, and is therefore close to the highest possible score. The global performance of the archive is 1.0.

4.3.2 Arrow Domain

The arrow domain uses $t_{max} = 1.0$ and $v_{max} = 1.0$. The archive plotted in the behavior space looks similar to the letter domain (see figure 4.13).

The first row on the variation %-axis cannot be filled because the tileset only contains 5 tiles. The second row requires only 1 tile to be used to create the entire level. This can only happen, when this is an X-tile. Thus only cell (4,1) is filled.

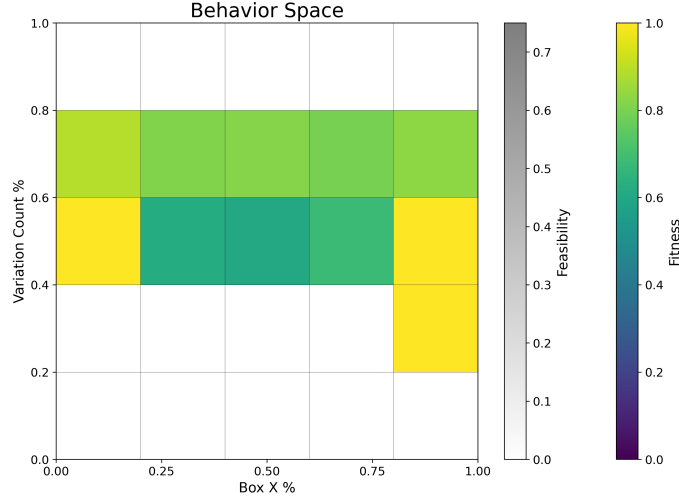


Figure 4.13: Behavior space of the archive generated for the Arrows domain.

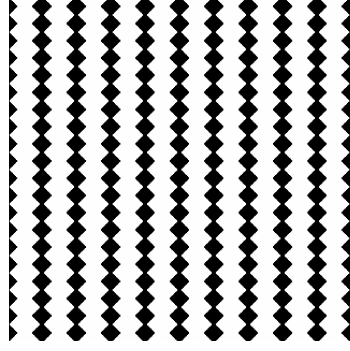
The third row shows all entries where only two tiles are used to create the level. We have made the input tileset such that this is only possible in two ways. Either the entire level consists entirely of X-tiles except for a single row along a border, which may be anything else. Alternatively, there are no X-tiles and the entire level consists of arrows pointing in alternating directions.

The genomes located in cells (1,2) (2,2) and (3,2) shown with a dark green color do not actually produce levels that land in these cells, since these levels are impossible to generate with the domain. Instead, they produce levels that land in the surrounding cells, but just happens to average to these cells.

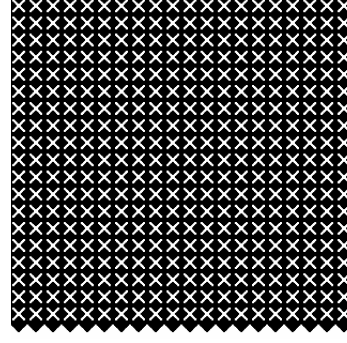
The fourth row consistently creates levels with three tiles, but does not score a perfect fitness value. The fifth row where four or all tiles are used in a level is impossible to reach given this domain.

Figure 4.14 shows examples of the kinds of levels that can be sampled from the archive.

There are only 8 reachable cells in this domain. Therefore, the highest possible QD score of the letter domain is $8/25 = 0.32$. Due to our behavior space partitioning and the approximation an individual's behavior based the average phenotype, the algorithm sometimes incorrectly identifies an individual as belonging to an logically unreachable cell. Thus, running CME with the Letter domain yields an archive with

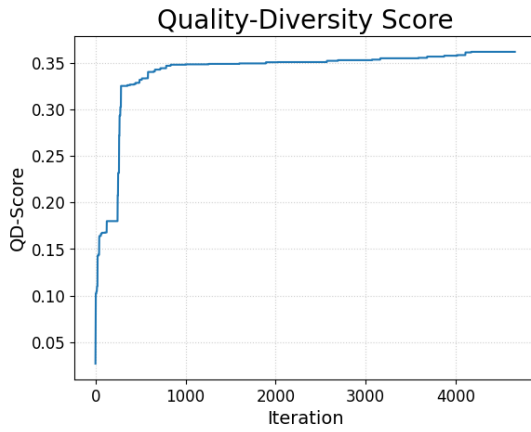


(a) A level generated by WFC from cell (0,2) in the archive.

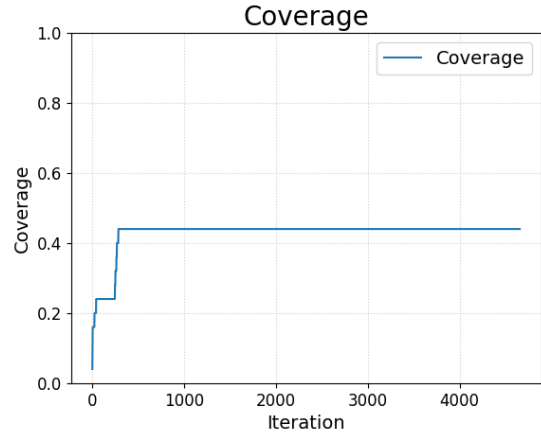


(b) A level generated by WFC from cell (4,2) in the archive.

Figure 4.14: Both levels use exactly 2 tiles.



(a) QD score of the archive.



(b) Coverage of the archive.

Figure 4.15: QD score and coverage of the archive for the Arrow domain.

a QD score of 0.38, which is higher than what should be possible (see figure 4.15). The global performance of the archive is 1.0.

The QD score is lower than the letter domain's. This indicates that this domain is more difficult for CME to explore and find high performing individuals within. Thus, we conclude that the arrow domain is a harder input domain than the letter domain.

4.3.3 Pokémon Domain

For the Pokémon domain, we set $t_{max} = 0.25$ and $v_{max} = 1.0$. The archive of the Pokémon domain is plotted in behavior space in figure 4.16.

Most entries in the archive has a fitness of around 0.8. This is likely due to the stochastic nature of the WFC algorithm when using a complicated input tileset.

Nothing has been explored with variation % less than 60%. In other words, the algorithm cannot find weights that use $\geq 60\%$ of the 97 tiles consistently.

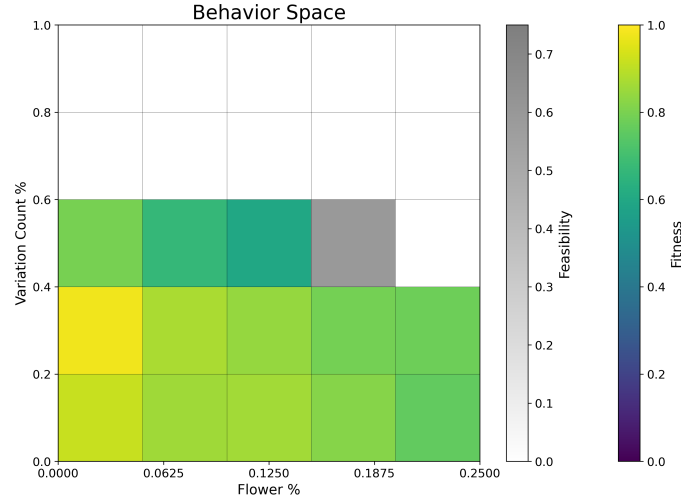


Figure 4.16: Behavior space of the archive generated for the Pokémon domain.



(a) A level from a genome sampled with less than 5% flower tiles.



(b) A level from a genome sampled with above 20% flower tiles.

Figure 4.17: Comparison of levels produced by genotypes in opposite ends of the flower-tile axis.

Cell (2,2) and (3,2) in figure 4.16 have a low fitness and (4,2) is infeasible. From this, we conclude that it is difficult for the algorithm to find genomes that have a high variation while also filling the level with flowers. This makes sense, since more flowers makes it difficult to have variation.

Using the *Archive Explorer* application to generate levels with this archive, it is clear that the flower % of the genotypes is reflected in the generated levels (see 4.17).

Figure 4.18 shows the feasible and infeasible population of the archive over time. The behavior space is explored during the first 250 initialization iterations and the following 250 mutation iterations before it settles on a feasible population size of 13 and a total population size of 14. The remaining 2000 iterations are spent optimizing the fitness of the archive. The two population lines show that the feasible population is populating the same cells as previously infeasible individuals.

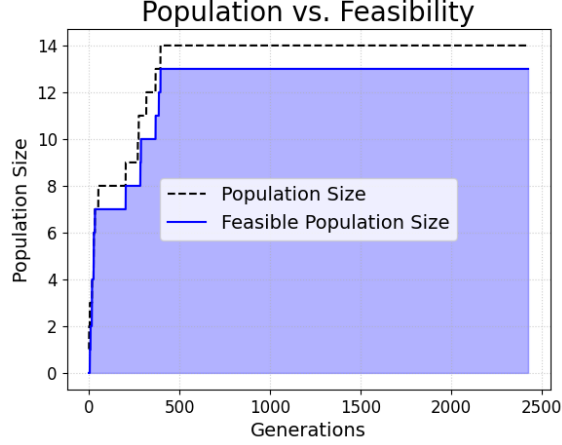
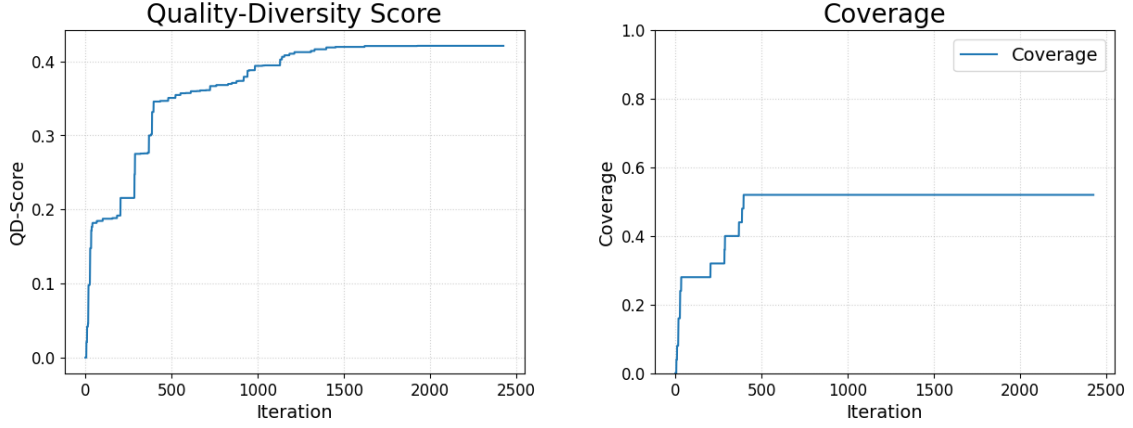


Figure 4.18: Feasibility graph of the evolution of the archive generated from the Pokémon domain.



(a) QD score of the archive.

(b) Coverage of the archive.

Figure 4.19: QD score and coverage of the archive for the Pokémon domain.

Using the Pokémon domain, the algorithm reaches a QD score of 0.42 and a coverage score of 0.52 (see figure 4.19). Further, it reaches a global performance of 0.98. The Pokémon therefore reaches a higher QD score than the Arrow domain but a worse global performance. As discussed earlier, the Arrow domain has more unreachable cells, which explains the lower QD score. Therefore, it makes sense to compare these domains using the Precision score (see section 3.5). The Precision score of the Arrow domain is 0.82 and the precision score of the Pokémon domain is 0.81. While the scores are very similar, we see that the Arrow domain does find better individuals in the cells it fills out compared to the Pokémon domain.

These results indicate that the amount of rules and tiles matters less than the fact that there are constraints present in the domain. The letter domain is therefore our best performing domain, while the Pokémon and Arrow domain are of similar difficulty.

In the rest of this chapter, we focus on optimizing and improving our approach

for the Pokémon domain, as this domain is our case study of a tileset used in a real-world scenario.

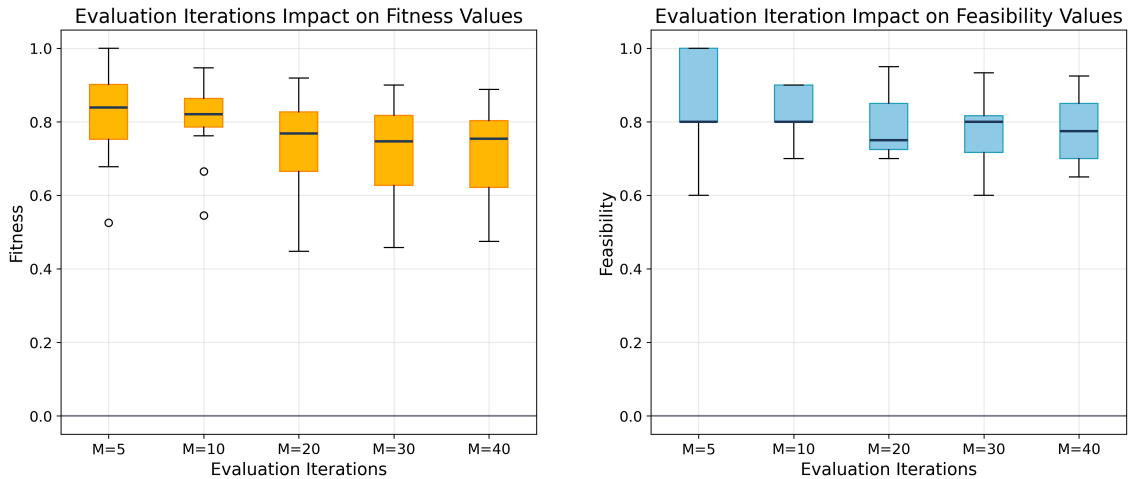
4.4 Evaluation Iterations

As mentioned in section 3.3.6, we expect larger values of M to more accurately represent the types of levels a genotype can generate. However, larger values for M takes longer to run. In the previous experiments, we set $M = 10$. This was a somewhat arbitrary choice. In this section we examine whether the value of M has an impact on the fitness and feasibility values of the archive.

For this experiment, we use the Pokémon domain. We set $D = 20$, $G = 250$, $I = 5000$, $C = 500$, $\tau = 0.75$, $k = 5$, $t_{max} = 0.25$, and $v_{max} = 1.0$.

We generate an archive using $M = 10$. We then reevaluate the archive using $M = 5$, $M = 10$, $M = 20$, $M = 30$, and $M = 40$. This way, we see if the same weights yield different fitness and feasibility values depending on how many evaluation iterations. The evaluations are seeded with the iteration number, meaning $M = 10$ uses the same first five results as $M = 5$ does.

Figure 4.20 shows the impact of M on the fitness and feasibility values. Each individual in the archive is data point in the box plots. $M = 5$ yields both higher and lower fitness values than $M = 10$. This indicates that $M = 5$ is especially susceptible to the stochastic noise of WFC. Sometimes, the first 5 levels indicate a much better genome than the next 5 would, and sometimes they indicate a much worse genome.



(a) Box plot of fitness values for different numbers of evaluation iterations.

(b) Box plot of feasibility values for different numbers of evaluation iterations.

Figure 4.20: The impact of different values of M on fitness and feasibility.

In contrast, $M = 20$ consistently yields lower fitness values than $M = 10$. This

Table 4.2: Evaluation Iteration impact on running times and QD score. All experiments use $I = 5000$ with no convergence.

M	Running time	Relative Increase	QD Score
5	214 s (~ 3.5 m)	N/A	0.446
10	291 s (~ 5 m)	36%	0.419
20	552 s (~ 9 m)	89.7%	0.400
30	766 s (~ 13 m)	38.8%	0.438
40	1098.1 s (~ 18 m)	43.2%	0.343

indicates that $M = 10$ is overly optimistic, since it saved the genomes that had 'lucky' levels, but for $M = 20$ it was less lucky.

For $M = 30$ and $M = 40$, the box plots look very similar to that of $M = 20$. Thus, for fitness at least, we conclude that $M = 20$ is sufficient to yield a good estimate for the phenotype.

The feasibility values show that $M = 5$ is overly optimistic compared to $M = 10$. It also has an outlier with a feasibility 0.6. For this to happen, 2 levels out of 5 have to be infeasible. In contrast for $M = 10$ 4 levels out of 10 have to be infeasible. Thus, an 'unlucky' seed impacts $M = 5$ much harder than $M = 10$.

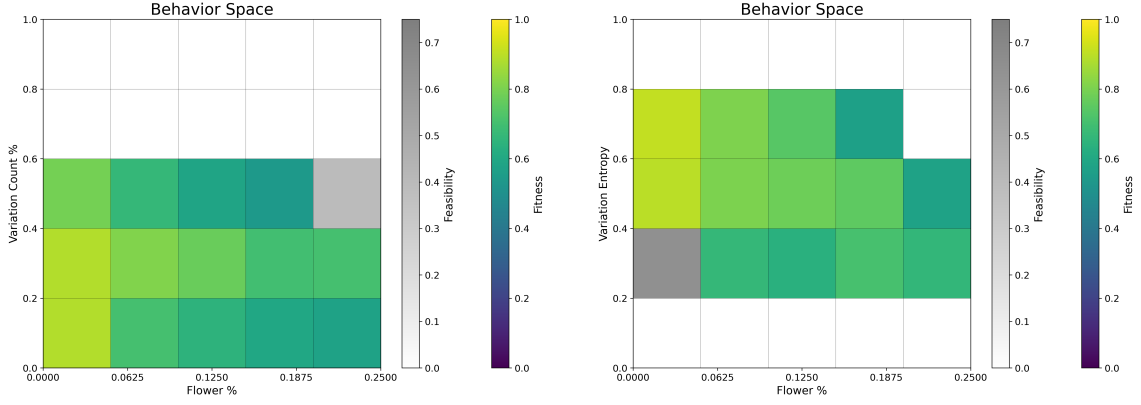
$M = 20$ generally scores lower on feasibility than $M = 10$, although one outlier actually scores higher, illustrating that the impact of the unlucky seed also affects $M = 10$. We conclude that the impact of $M = 30$ and $M = 40$ are negligible compared to $M = 20$.

Table 4.2 shows the impact of M on the running time and QD score of the archive. The running time increases as M increases, though the increase is non-linear. This is partly due to the fact that we multi-thread the evaluations. The computer on which we run the experiments has a 12-core CPU, so 12 evaluations can run at the same time.

With $M = 20$, 5000 iterations takes under 10 minutes to run, and, as seen in figure 4.20, still produces reliable results. Thus, for future experiments, we choose a value of $M = 20$.

4.5 Variation Behavior

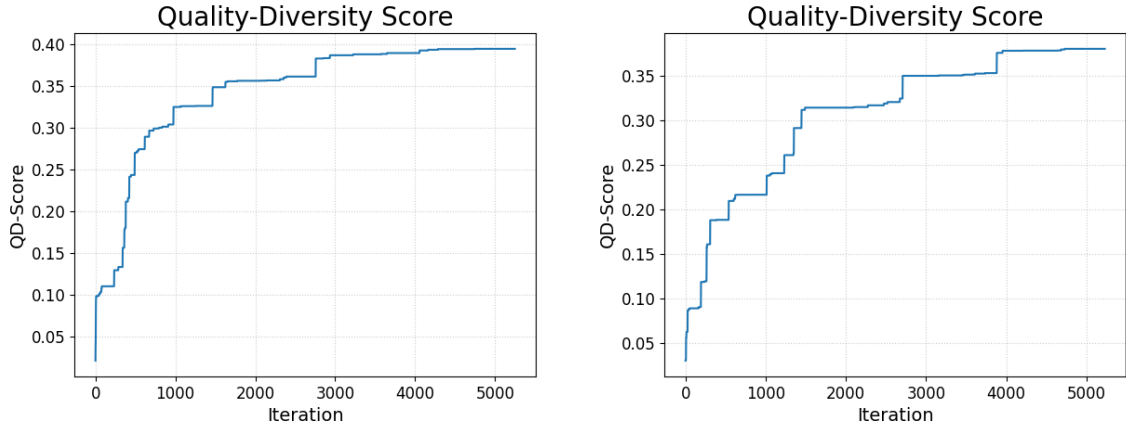
As mentioned in section 3.3.6, we implement two ways of calculating *Variation %*: *Unique Count* and *Entropy*. For all experiments so far, we have used the *Unique Count* as our variation behavior. In this section, we run experiments of the two



(a) Behavior space when measuring variation with unique tile count.

(b) Behavior space when measuring variation with entropy.

Figure 4.21: Behavior space comparison of the two variation behaviors.



(a) QD score of the archive when using the *unique count* heuristic.

(b) QD score of the archive when using the *entropy* heuristic.

Figure 4.22: QD score of the two variation behaviors.

variation behaviors and compare them. The point of this experiment is to test whether changing how we measure variation will impact the QD score of our archive.

For these experiments, we set $D = 20$, $M = 20$, $G = 250$, $I = 5000$, $C = 500$, $\tau = 0.75$, $k = 5$, $t_{max} = 0.25$, and $v_{max} = 1.0$.

When we compare the behavior space diagrams in figure 4.21 side by side, we observe that all entries have shifted one row. However, we also see roughly the same number of entries in the archive, indicating that changing how variation is calculated does not impact the efficiency of illumination of the algorithm.

In figure 4.22 we observe that both archives reach roughly the same QD score of 0.383 and 0.382. This indicates that with either way of calculating variation, we reach the same fitness across the archive.

With this experiment, we conclude that it does not matter which method we use to calculate variation. In the following experiments, we will continue using the

unique count heuristic.

4.6 Mutation Strategies

As highlighted in section 3.3.5, there are many ways to mutate a genome. For all experiments so far, we use the *All Tiles* mutation (see table 3.2).

In this section, we present results from using the different mutation strategies. For all experiments, we use the Pokémon domain and set $D = 20, M = 20, G = 250, I = 5000, C = 500, \tau = 0.75, k = 5, t_{max} = 0.25$, and $v_{max} = 1.0$.

The CMA-CME mutation strategy requires another set of parameters. We use $e_o = 2$, $e_f = 5$, and $e_d = 3$ to denote the amount of optimization, feasibility, and random direction emitters that will be used. Further, we set a starting step size of $\sigma = 0.14$, as it showed good initial results. The starting step size is used as the starting step size for the CMA engine. Lastly, we also include a stagnation threshold $\rho = 30$. If an emitter has not emitted a genome that is added to the archive for the last ρ iterations, it is reset with a new *mean genome*. These values are semi-arbitrary, based on only a few runs of trial-and-error. Further work would be required to establish good values for these parameters and fully show the potential of *CMA-CME*.

Table 4.3 shows the results for the four mutation strategies. Mutating only the special tile covers 20 % of the behavior space, and the solutions found this way are generally low performing, which is reflected in the QD score 0.133. For this strategy to function ideally, the uniform sampling in the first G iterations has to cover the whole variation axis. Then mutating only the special tile might be able to cover the flower axis completely. Looking at the behavior space of the archive (see figure 4.23c), the mutation strategy does not fully cover the special tile axis, presumably because the other weights are too high relative to the flower tile.

Table 4.3: Results for different mutation strategies. Refer to table 3.2 for details on mutation strategies.

Mutation	QD score	Global Perf.	Coverage	Running Time
All Tiles	0.383	0.926	0.52	950.5 s (~ 16 m)
1/3 Tiles	0.433	0.895	0.6	1071.4 s (~ 18 m)
Special Tile	0.133	0.776	0.2	619.8 s (~ 10 m)
CMA-CME	0.387	0.850	0.6	1524.4 s (~ 25 m)

The *All Tiles*, *1/3 Tiles* and *CMA-CME* strategies all perform relatively well on

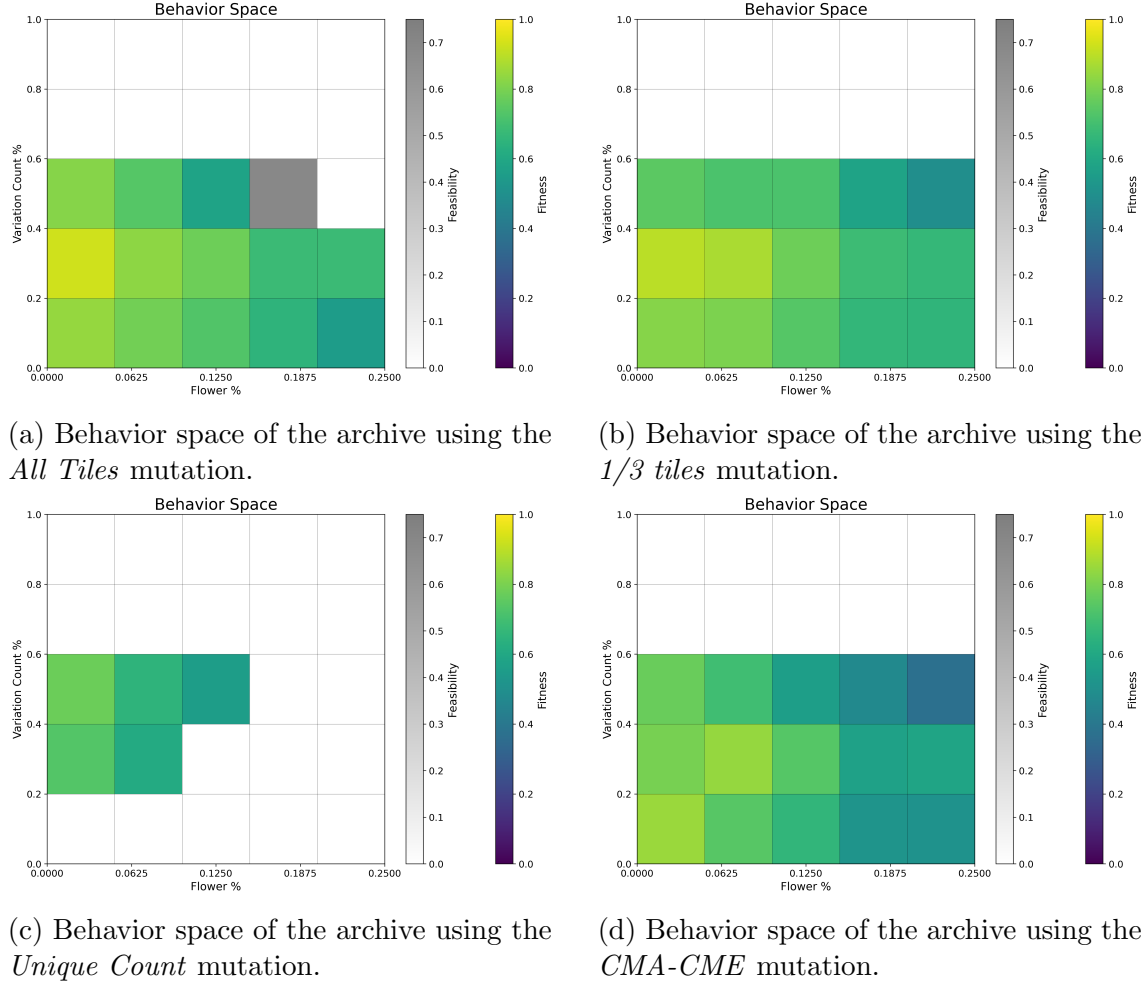


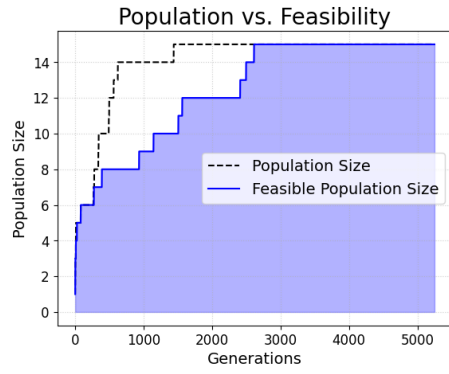
Figure 4.23: Behavior space plot of the four mutation strategies.

most metrics. *All Tiles* has the highest global performance, but a lower coverage and QD score. Note that coverage and QD score are highly correlated. The *CMA-CME* strategy performs comparably well on coverage and QD score, but takes about 50 % longer to run. In fact, *CMA-CME* was the only of the mutation strategies to not converge in at least 5250 iterations.

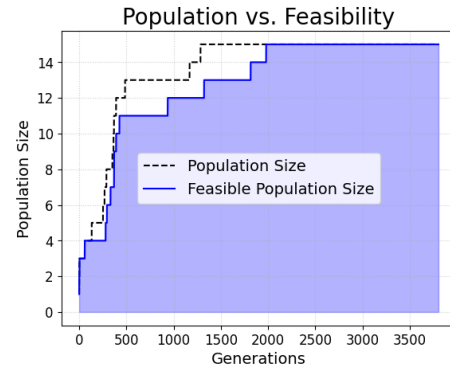
The best overall performer was the *1/3 tiles* mutation strategy, since it has the highest coverage, QD score, and is close to the fitness value of the *All Tiles* mutation strategy. Further, it only takes a few minutes longer to run than the *All Tiles* mutation.

The three best mutation strategies also have similar behavior spaces (see figure 4.23). None of them can fill the variation axis above 60 %. Further, they all have difficulty finding high fitness genomes in the (3,2) and (4,2) cell. They all find well performing genomes in the same area (around (0,1) and (1,1)).

CMA-CME quickly finds a large population of infeasible individuals, and then spends many iterations making them feasible. This can be seen in figure 4.24a. In contrast, the *1/3 Tile* mutation strategy seems to generate feasible and infeasible



(a) Feasible and Infeasible Population of *CMA-CME* over time.



(b) Feasible and Infeasible Population of $1/3$ Tiles over time.

Figure 4.24: Feasible and Infeasible Population in the archive over time.

solutions at roughly the same pace.

5 Results

In this section we relate the experiments from chapter 4 to our research questions. We evaluate whether we succeed by summarizing our findings and discussing the results.

5.1 Illumination of Behavior Space

Our first research question is: Can the mutation of weights for the Wave Function Collapse algorithm (WFC) illuminate a behavior space? This addresses the diversity aspect of quality-diversity (QD), which is often a desirable trait among procedural content generation algorithms. In the context of our research, we want to know if modifying the input weights to WFC can generate levels that exhibit different behaviors. We measure the percentage of the behavior space archive that is filled in terms of *coverage*.

In our experiments we have defined two behavior metrics that we differentiate our levels on. One axis of our behavior space considers the variation of tiles in the generated levels. Another axis considers the share of a *special tile* in the generated levels. Our goal is to find genomes that map to phenotypes that cover the entire behavior space.

In chapter 4, we introduce 3 domains. The first domain is the Letter domain, a domain without any constraints, i.e. all tiles can be adjacent to each other. The second domain is the Arrow domain, introducing constraints between the tiles. The third domain is the Pokémon domain, using a tilemap from *Pokémon Fire Red* (Game Freak, 2004). The Pokémon domain has many more tiles and rules than the two other domains, and thus is our most complex domain.

According to experiments in section 4.3, Constrained MAP-Elites (CME) explores all reachable cells for the Letter and Arrow domain. In the more difficult Pokémon domain, CME explores 60% of its behavior space.

In section 4.5, we present experiments with different calculation strategies for the variation behavior. With this, we examine if different behavior calculations impact the coverage of CME. The results show that these particular behaviors do not impact the coverage, although the explored behavior space is translated upwards (see figure 4.21).

This indicates that CME is not always able to illuminate the full behavior space with WFC as the phenotype mapping. We hypothesize that because WFC is a stochastic algorithm, only modifying the input weights cannot reach 100% variation.

In section 4.6, we experiment with different mutation strategies to test, if the

choice of mutation has an impact on the illumination of the behavior space. The results show that an implementation of CME that uses covariance-matrix adaption (CMA-CME) to mutate the weights illuminates about as much of the behavior space as mutation strategies that use normal sampling.

From our experiments we conclude that CME can explore wide in the behavior space. Therefore the mutation of weights for the WFC algorithm can illuminate a behavior space. However, we also conclude that it cannot cover all parts of the behavior space. Further work would be needed to establish whether this is a shortcoming of our behavior space partitioning, choice of genotype, or mutation strategy.

5.2 Reliability of Wave Function Collapse

Our second research question is: Can Quality-Diversity (QD) improve the reliability of the Wave Function Collapse algorithm (WFC)? This addresses the issue of contradictions within WFC, that makes it unreliable to run to completion.

To address this research question, we consider the Pokémon domain, as it is the only domain that can reach contradictions. This allows us to measure and evaluate feasibility.

For all experiments, we set a feasibility threshold of 0.75. This means that for a genome to be considered feasible, 75% of the levels generated by WFC must be complete.

In our experiments, the feasible population of the archive increases over time (see figure 4.24b). This indicates that Constrained MAP-Elites (CME) can produce infeasible genomes and eventually find feasible individuals for cells with currently infeasible individuals.

We could have simply discarded any genome that does not meet the feasibility threshold. However, we use CME rather than MAP-Elites to directly address the reliability of WFC. The hope is that infeasible genomes may become feasible over time. However, we have no metric that tracks whether the newly feasible genomes originate from the infeasible population, rather than the feasible population. Therefore, we cannot conclude that the infeasible population directly helps with the feasibility constraint.

We conclude that CME successfully generates weights that reliably avoid contradictions in WFC at least 75% of the time. Therefore, QD methods can improve the reliability of the WFC algorithm by up to 75 % or more. Further work would be needed to establish how far this threshold can be pushed.

5.3 Consistency of Wave Function Collapse

Our third research question is: Can QD improve the consistency of the WFC algorithm? This question addresses the stochastic results produced by WFC. We want to establish a higher degree of statistical predictability in the characteristics of a level generated from a genome. We address this by generating a set of weights and measure how consistently WFC produces levels with similar characteristics using these weights.

The *fitness* function of our CME implementation measures the average euclidean distance in behavior space between phenotypes generated from the same genome. A smaller average distance yields a higher fitness score. We consider genomes that produce levels that land in the same area of behavior space *consistent*. Therefore, a high fitness value corresponds to high consistency.

The QD score measures the average fitness across all cells in the archive. Thus, a high QD score represents a high likelihood that an individual randomly sampled from the archive has a high fitness score.

Our experiments in section 4.3 show that running CME with the Letter domain yields an archive with a QD score of 0.65. It should be noted that the QD score is punished by not being able to illuminate the entire search space. The way our behavior space is partitioned makes this impossible for the Letter domain, which means that a perfect QD score is impossible. The theoretical best QD score for the Letter domain is 0.68. The relatively high QD score shows that it finds consistent weights that can create many types of levels.

The global performance of the archive is 1.0. This shows that CME is able to find perfectly consistent genomes.

Running CME with the Arrow domain yields an archive with a QD score of 0.38. The theoretical best QD score for the arrow domain is 0.32. This does not make sense analytically, but is an artifact of averaging inaccuracies, leading CME to incorrectly place some individuals in the wrong cells. Taking this into account, the archive for the Arrow domain reaches a QD score of 0.29.

This domain finds fewer genomes that are as consistent, compared to the Letter domain. Thus, introducing local constraints to WFC seems to have a significant impact on the ability to create consistent genomes. Note that this is also correlated with how much of the behavior space is illuminated.

The global performance of the archive is 1.0. This shows that while CME is able to find perfectly consistent genomes.

Running CME with the Pokémon domain yields a QD score of 0.43. The global performance of is around 0.9. Although not perfect, this shows that CME is able to find consistent individuals, even in our most complex domain. The plot of the

behavior space (see figure 4.16) shows that some cells have a high fitness score, some cells have a lower fitness score, one cell is infeasible, and some cells have not been covered by CME. We do not have an estimate for the best theoretical QD score for the Pokémon domain.

It should be noted that we have run more experiments to improve the results of the Pokémon domain. Experimenting with behavior space partitioning and mutation strategies with the Letter and Arrow domain might significantly improve their scores.

We conclude that it is possible to find some very consistent weights for WFC using QD methods. We also conclude that creating some behavioral niches consistently is very difficult by manipulating only the input weights.

6 Conclusion

This thesis has presented a Search-Based Procedural Content Generation (SB-PCG) approach to improve the inherent limitations of the Wave Function Collapse algorithm (WFC). Problematically, WFC can arrive at contradictions, where a level cannot be generated to completion. Further, WFC is a stochastic algorithm, making generating similar levels challenging.

This relates to a central problem of PCG: Boredom vs. chaos. Too similar output is boring, while completely random output is not useful. We use the quality-diversity algorithm Constrained MAP-Elites (CME) to generate an archive of input weights to WFC. The archive is a behavior space partitioning, meaning that cells in the archive exhibit diversity in the behavioral axes. This addresses the boredom aspect. We use consistency as the fitness metric. This addresses the chaos aspect.

Our results indicate that by mutating only the input weights for WFC, it is possible to illuminate much of a behavior space, though not necessarily the entire behavior space. Our results further indicate that CME is able to generate feasible individuals and improve their feasibility over time. CME is also able to improve the consistency of the levels generated by a set of input weights. We also find that not all behavioral niches can be filled and that some behavioral niches have less consistency than others.

6.1 Contributions

With this project, we developed a command line interface (CLI) tool called *InfinimonCLI* to generate an archive of input weights to WFC. Further, we provide an interactive tool called *Archive Explorer* to explore which levels can be generated from genomes in the archive.

This thesis also presents the ground work for Covariance-Matrix Adaption MAP-Elites adapted to work with Constrained MAP-Elites; an algorithm we have called *CMA-CME*.

6.2 Future Work

Although our results indicate that quality-diversity methods can improve the consistency and reliability of WFC, there are several interesting directions to take this work.

6.2.1 Mutation Strategies

This thesis introduces Covariance-Matrix Adaption for Constrained MAP-Elites (CMA-CME), but further work is required to establish its full potential. CMA dynamically changes both its search direction and step size and has been developed to more efficiently traverse a search space than normal sampling. CMA-CME did not perform better than CME with uniform sampling in our experiments, but we expect that this is due to the setup and parameters for these experiments. Further work could examine more emitter types and the distribution of these within the population of emitters. This could illuminate more of the behavior space and provide faster convergence. Experimenting with a dynamic set of emitters that adapt to the evolution of the archive while exploring could improve the results of CMA-CME even further.

However, other mutation strategies using normal sampling might lead to better CME performance. The results show that mutating only one third of the weights in each iteration leads to a better QD score in the archive than mutating all of the weights. Further work could explore other strategies for mutating the weights, e.g. using a ranking system to select which weights to mutate.

6.2.2 Other Genotypes

Brellas, 2021 uses the input image to WFC as the genotype. We choose the input weights as the genotype. Further work could experiment with mutating other parts of the input, such as the input tiles themselves or the adjacency rules.

6.2.3 Mixed-Initiative Design

We have introduced the *Archive Explorer* and *InfinimonCLI* tool. These tools serve as vehicles for our research, but further work could conduct a user study with content designers and adapt these tools to support a mixed-initiative workflow.

Bibliography

- Acton, M. (2014, September 30). *CppCon 2014: Mike acton "data-oriented design and c++"*. Retrieved April 24, 2026, from <https://www.youtube.com/watch?v=rX0ItVEVjHc>
- Backus, K. (2017). Managing output: Boredom vs. chaos [OCLC: 990267798]. In *Procedural generation in game design*. Taylor & Francis, CRC Press.
- Braben, D., & Bell, I. (1984). *Elite*.
- Brellas, I. (2021). *Combining wave function collapse and evolutionary algorithms for controlled content generation* [Master's Thesis]. University of Malta. Retrieved February 6, 2026, from <https://www.um.edu.mt/library/oar/handle/123456789/107911>
- Bromiley, P. A., Thacker, N. A., & Bouhova-Thacker, E. (2004). Shannon entropy, renyi entropy, and information. *Statistics and Inf. Series (2004-004)*, 9(2004), 2–8. Retrieved March 11, 2026, from <https://www.academia.edu/download/52511411/2004-004.pdf>
- Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (Third edition, global edition). Pearson.
- CabbageCat. (2022, October 9). *CMA-ME: Using CMA-ES to guide MAP-elites* [CabbageCat's blogs]. Retrieved April 23, 2026, from <https://szhaovas.github.io/2022-10-09-cmame/>
- Desmos. (n.d.). *Desmos graphing calculator* [Desmos]. Retrieved May 14, 2026, from <https://www.desmos.com/calculator>
- Donald, M. (2020, July 31). *Superpositions, sudoku, the wave function collapse algorithm*. Retrieved April 25, 2026, from <https://www.youtube.com/watch?v=2SuvO4Gi7uY&t=402s>
- Edmund McMillen and Florian Himsl. (2011). *The binding of isaac*.
- Efros, A., & Leung, T. (1999). Texture synthesis by non-parametric sampling. *Proceedings of the Seventh IEEE International Conference on Computer Vision*, 1033–1038 vol.2. <https://doi.org/10.1109/ICCV.1999.790383>
- Fontaine, M. C., Togelius, J., Nikolaidis, S., & Hoover, A. K. (2020). Covariance matrix adaptation for the rapid illumination of behavior space. *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, 94–102. <https://doi.org/10.1145/3377930.3390232>
- Game Freak. (2004). *Pokémon fire red*.
- Google. (n.d.). *Gemini – Your AI assistant from Google* [Gemini]. Retrieved May 21, 2026, from <https://gemini.google/dk/about/?hl=da>

- Gravina, D., Khalifa, A., Liapis, A., Togelius, J., & Yannakakis, G. N. (2019). Procedural content generation through quality diversity. *2019 IEEE Conference on Games (CoG)*, 1–8. <https://doi.org/10.1109/CIG.2019.8848053>
- Grey, D. (2017). When and why to use procedural generation [OCLC: 990267798]. In *Procedural generation in game design*. Taylor & Francis, CRC Press.
- Gumin, M. (2016, September). *Wave function collapse algorithm* (Version 1.0) [original-date: 2016-09-30T11:53:17Z]. Retrieved January 28, 2026, from <https://github.com/mxgmn/WaveFunctionCollapse>
- Ha, D. (2017). A visual guide to evolution strategies. Retrieved March 25, 2026, from <https://blog.otoro.net/2017/10/29/visual-evolution-strategies/>
- Hello Games. (2016). *No man's sky*.
- Hoffmann, C. M., & Joan-Arinyo, R. (2005). A brief on constraint solving. *Computer-Aided Design and Applications*, 2(5), 655–663. <https://doi.org/10.1080/16864360.2005.10738330>
- JetBrains. (n.d.). *Full line code completion — JetBrains rider* [JetBrains rider help]. Retrieved May 21, 2026, from https://www.jetbrains.com/help/rider/Full_Line_code_completion.html
- Karth, I., & Smith, A. M. (2017). WaveFunctionCollapse is constraint solving in the wild. *Proceedings of the 12th International Conference on the Foundations of Digital Games*, 1–10. <https://doi.org/10.1145/3102071.3110566>
- Khalifa, A., Lee, S., Nealen, A., & Togelius, J. (2018). Talakat: Bullet hell generation through constrained map-elites. *Proceedings of the Genetic and Evolutionary Computation Conference*, 1047–1054. <https://doi.org/10.1145/3205455.3205470>
- Kimbrough, S. O., Koehler, G. J., Lu, M., & Wood, D. H. (2008). On a feasible–infeasible two-population (FI-2pop) genetic algorithm for constrained optimization: Distance tracing and no free lunch. *European Journal of Operational Research*, 190(2), 310–327. <https://doi.org/10.1016/j.ejor.2007.06.028>
- Lehman, J., & Stanley, K. O. (2008). Exploiting open-endedness to solve problems through the search for novelty. *ALIFE*, 65, 329–336. Retrieved March 4, 2026, from <https://www.academia.edu/download/25396037/0262287196chap43.pdf>
- Mackworth, A. K. (1977). Consistency in networks of relations. *Artificial Intelligence*, 8(1), 99–118. [https://doi.org/10.1016/0004-3702\(77\)90007-8](https://doi.org/10.1016/0004-3702(77)90007-8)
- Merrell, P. C. (2009). *Model synthesis* [Doctoral dissertation, The University of North Carolina at Chapel Hill]. Retrieved January 28, 2026, from <https://search.proquest.com/openview/5e7eae9aa03355cc1ff71075ec005851/1?pq-origsite=gscholar&cbl=18750>

- Microsoft. (n.d.). *BitArray class (system.collections)*. Retrieved April 22, 2026, from <https://learn.microsoft.com/en-us/dotnet/api/system.collections.bitarray?view=net-10.0>
- Mitchell, M. (1998). *An introduction to genetic algorithms*. MIT press. Retrieved December 29, 2025, from <https://books.google.com/books?hl=en&lr=&id=0eznlz0TF-IC&oi=fnd&pg=PP9&dq=genetic+algorithm&ots=sjoH1-1cOk&sig=VvwfjzIf4r5reVUq4H4wbDIqC34>
- Mohr, R., & Henderson, T. C. (1986). Arc and path consistency revisited. *Artificial Intelligence*, 28(2), 225–233. [https://doi.org/10.1016/0004-3702\(86\)90083-4](https://doi.org/10.1016/0004-3702(86)90083-4)
- Mojang Studios. (2011). *Minecraft*.
- Montoya, O., Srb, F., Grbic, D., & Risi, S. (2025). CPPN2wfc: Extending wave function collapse to generate globally coherent content. *Proceedings of the Genetic and Evolutionary Computation Conference*, 1255–1263. <https://doi.org/10.1145/3712256.3726401>
- Mossmouth, Derek Yu, Blitworks. (2008). *Spelunky*.
- Mouret, J.-B., & Clune, J. (2015, April 20). Illuminating search spaces by mapping elites. <https://doi.org/10.48550/arXiv.1504.04909>
- O’Leary, M. (2026, March 21). *Mewo2/oisin* [original-date: 2017-05-23T14:35:25Z]. Retrieved May 12, 2026, from <https://github.com/mewo2/oisin>
- Oskar Stålberg. (2021). *Townscaper*.
- Pounce Light. (2024). *Tiny glade*.
- Sandhu, A., Chen, Z., & McCoy, J. (2019). Enhancing wave function collapse with design-level constraints. *Proceedings of the 14th International Conference on the Foundations of Digital Games*, 1–9. <https://doi.org/10.1145/3337722.3337752>
- selfsame. (2016). *Unity-wave-function-collapse by selfsame* [Itch.io]. Retrieved March 11, 2026, from <https://selfsame.itch.io/unitywfc>
- Shaker, N., Togelius, J., & Nelson, M. J. (2016). *Procedural content generation in games*. Springer International Publishing. <https://doi.org/10.1007/978-3-319-42716-4>
- The Matplotlib Development Team. (2026, April 24). *Matplotlib: Visualization with python* (Version v3.10.9). Zenodo. <https://doi.org/10.5281/ZENODO.19716234>
- Togelius, J., Yannakakis, G. N., Stanley, K. O., & Browne, C. (2011). Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3), 172–186. Retrieved January 28, 2026, from <https://ieeexplore.ieee.org/abstract/document/5756645/>